ATM33/e Series

Reference Manual (Preliminary)

SUMMARY: This reference manual provides detailed functional description of ATM33/ATM33e Wireless SoC Series, and how to implement solutions using the ATM33/ATM33e series SoC.

CONFIDENTIAL



Atmosic™

Table of Contents

Acronyms and Abbreviations 5

1. Overview 8

2. Functional Description 9

2.1 MCU and Memory 9

2.1.1 Clocks 11

RC Oscillator Measurement 12

WRPR 13

2.1.2 APB Subsystem 14

2.1.3 Reset 15

2.1.4 Wakeup Timer 15

2.1.5 General Purpose Timers 15

2.1.6 Interrupts 16

2.1.7 Power Modes 19

2.1.8 OTP Memory 20

2.1.9 Debug Access 22

2.1.10 Trace Port Interface Unit 22

2.1.11 Boot Flow 22

2.2 Security 23

2.2.1 Security Goals 24

Unique ID 24

Attestation 24

Secure Boot 24

Secure Update 25

Anti-Rollback 25

Isolation and Interaction 25

Secure Lifecycle 25

2.2.2 Atmosic Security Blocks 26

AES-128 & AES-256 26

SHA-256 & HMAC-256 27

TRNG (True Random Number Generator) 28

Secure Data Area 28

Private Key Storage 29

Secure Counters 30

Secure Journal 30

Protected Data 30

Boot Blocks 30

CONFIDENTIAL

RRAM/ROM Protections 30
Secure Counters 31
Secure Debug 32
Secure DMA 32
2.2.3 ARM TrustZone-M 33
Secure and Non-Secure Modes 33
Implementation Defined Attribution Unit (IDAU) 34
Security Arbitration Unit (SAU) 34
Memory Protection Controllers 35
Peripheral Protection Controllers 36
2.3 Power Management Unit (PMU) 39
2.4 PMU Configurations 43
2.4.1 Supported Battery Types 45
2.4.2 Battery Level 45
2.4.3 Internal or External I/O Supply 45
2.4.4 System Brownout Threshold 46
2.4.5 Harvesting Type (ATM3330e only) 46
2.4.6 VSTORE maximum threshold (ATM3330e only) 47
2.5 Sensor Hub (SHUB) 57
2.5.1 Event Schedule 59
2.5.2 Data Comparison 59
2.5.3 External Flash Control 61
2.5.4 Storage in SRAM 61
2.5.5 Advertisement with SHUB 62
2.6 Bluetooth LE Radio 63
2.6.1 Link Controller 63
2.6.2 LC Module Description 64
2.6.3 Modem 65
Receiver 65
Transmitter 65
2.7 Wakeup Receiver 66
2.8 Pin Multiplexing 68
2.9 Program Sequencer (PSEQ) 75
2.10 Peripherals and I/O 78
2.10.1 GPIO 78
Controlling input enable and pull up enable 79
2.10.2 Inter-Integrated Circuit (I2C) 80
2.10.3 Serial Peripheral Interface (SPI) 82
]2.10.4 Quad serial peripheral interface (QSPI) 85
2.10.5 Universal Asynchronous Receiver/Transmitter (UART) 87

CONFIDENTIAL

2.10.6 Pulse Density Modulation (PDM) Digital Microphone 87
2.10.7 Pulse Width Modulation (PWM) 89
2.10.8 Quadrature Decoder for Mouse Inputs (QDEC) 91
2.10.9 Keyboard Scan Matrix Controller (KSM) 92
2.10.10 Analog Comparator 92
2.11 General-purpose ADC (GADC) 93
2.12 Register Specification Document 96
Reference Documents 97
Revision History 98

CONFIDENTIAL

List of Figures

Figure 2-1 Functional Block Diagram
Figure 2-2 Boot Flow from PWD Deasserted/VBAT Power on to SW Bootloader
Figure 2-3 Voltage Rail Regulation Sequence
Figure 2-4 Voltage Rail Regulation After PWD Deassertion
Figure 2-5 PMU Block Diagram
Figure 2-6 Regular Battery, Internal I/O Supply, No Harvesting Capability
Figure 2-7 Regular Battery, Internal I/O Supply, RF Harvesting Source
Figure 2-8 Regular Battery, Internal I/O Supply, Non-RF Harvesting Source
Figure 2-9 Regular Battery, External I/O Supply, No Harvesting Capability
Figure 2-10 Regular Battery, External I/O Supply, RF Harvesting Source
Figure 2-11 Regular Battery, External I/O Supply, Non-RF Harvesting Source
Figure 2-12 Li-ion Battery or Storage Device on VBATLI, Internal I/O Supply, No Harvesting Capability
Figure 2-13 Li-Ion Battery or Storage Device on VBATLI, Internal I/O Supply, RF Harvesting Source
Figure 2-14 Li-Ion Battery or Storage Device on VBATLI, Internal I/O Supply, Non-RF Harvesting Source
Figure 2-15 Sensor Hub Block Diagram
Figure 2-16 Bluetooth LE Radio Block Diagram
Figure 2-17 Wakeup Receiver Application Block Diagram
Figure 2-18 Wakeup Receiver Block Diagram
Figure 2-19 Application ADC Block Diagram

List of Tables

Table 1-1 ATM33/e Devices and Packages
Table 2-1 System Memory Map

Table 2-2 List of APB Slave Cores
Table 2-3 List of Maskable Interrupts
Table 2-4 Secure RRAM Data Structures
Table 2-5 OTP Debug Settings
Table 2-6 Secure Alias Example
Table 2-7 CPU or DMA Security Mode to Access Secure or Nonsecure Memory Block
Table 2-8 CPU or DMA Security Mode to Access Secure or Nonsecure Peripheral
Table 2-9 AHBNSPPCEXP1 Register
Table 2-10 APBNSPPCEXP0 Register
Table 2-11 APBNSPPCEXP1 Register
Table 2-12 APBNSPPCEXP2 Register
Table 2-13 PMU External Pins
Table 2-14 PMU Configuration Fields
Table 2-15 PMU Supported Configuration
Table 2-16 Set VSTORE Maximum Threshold
Table 2-17 All Supported PMU Configurations
Table 2-18 Total Number of I/Os
Table 2-19 Multiplexed Pin Table
Table 2-20 Pin Selection Control to Pin Mapping
Table 2-21 Interrupt Registers
Table 2-22 PDM Block Specifications
Table 2-23 Application ADC Electrical Parameters
Table 2-24 ADC Channels Summary

Acronyms and Abbreviations

Acronyms	Definition
ADC	Analog to Digital Converter
AES	Advanced Encryption Standard
AHB	Advanced High performance Bus
AMBA	Advanced Microcontroller Bus Architecture
ATM33	ATM3325 ATM3330
ATM33e	ATM3330e/
ATM33/e	ATM33/ATM33e
CBC	Cipher Block Chaining
CFB	Cipher Feedback
CTR	Counter

CONFIDENTIAL

Acronyms	Definition
DMA	Direct Memory Access
DSP	Digital Signal Processing
ECB	Electronic Code Book
EVK	Evaluation Kit
FPU	Floating Point Unit
GADC	General purpose Analog to Digital Converter
GFSK	Gaussian Frequency Shift Keying
GPIO	General Purpose Input Output
HMAC	Hashed Message Authentication Code
IV	Initialization Vector
IoT	Internet of Things
KSM	Keyboard Scan Matrix
LDO	Low Drop Out
MCU	Microcontroller Unit
MDM	Modem
MPC	Memory Protection Controller
NVM	Non-Volatile Memory
OTP	One Time Programmable
OFB	Output Feedback
PDM	Pulse Density Modulation
PMU	Power Management Unit
PPC	Peripheral Protection Controller
PSEQ	Program Sequencer
PWD	Power Down
QDEC	Quadrature Decoder
QFN	Quad Flat No-lead

CONFIDENTIAL

Acronyms	Definition
QSPI	Quad Serial Peripheral Interface
RAM	Random Access Memory
RISC	Reduced Instruction Set Computer
ROM	Read Only Memory
RRAM	Resistive Random-Access Memory
RSSI	Received Signal Strength Indicator
SDK	Software Development Kit
SHA	Secure Hash Algorithm
SoC	System on Chip
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
SW	Software
SWD	Serial Wire Debug
TEG	Thermal Electric Generators
TRNG	True Random Number Generator
WFI	Wait for Interrupt
WLCSP	Wafer Level Chip Scale Package

CONFIDENTIAL

1. Overview

The ATM33/e is a series of extreme low-power Bluetooth® 5.3 SoC devices. This Bluetooth Low Energy SoC integrates a Bluetooth 5.3 compliant radio with ARM® Cortex® M33F application processor, 128 KB RAM, 64 KB ROM, 512 KB NVM, with ARM® TrustZone® enabled security features, and state-of-the-art power management to enable maximum lifetime in battery-operated devices.

This reference manual describes in detail on how to implement solutions using the ATM33/e series, which consist of:

Device	Package
ATM33e Series	ATM3330e 7x7 mm QFN package
ATM33 Series	ATM3330 7x7 mm QFN package
	ATM3325 5x5 mm QFN package
	ATM3325 5x5 mm QFN with extended storage package

Table 1-1 ATM33/e Devices and Packages

For related documents, please see the Reference Documents section.

CONFIDENTIAL

2. Functional Description

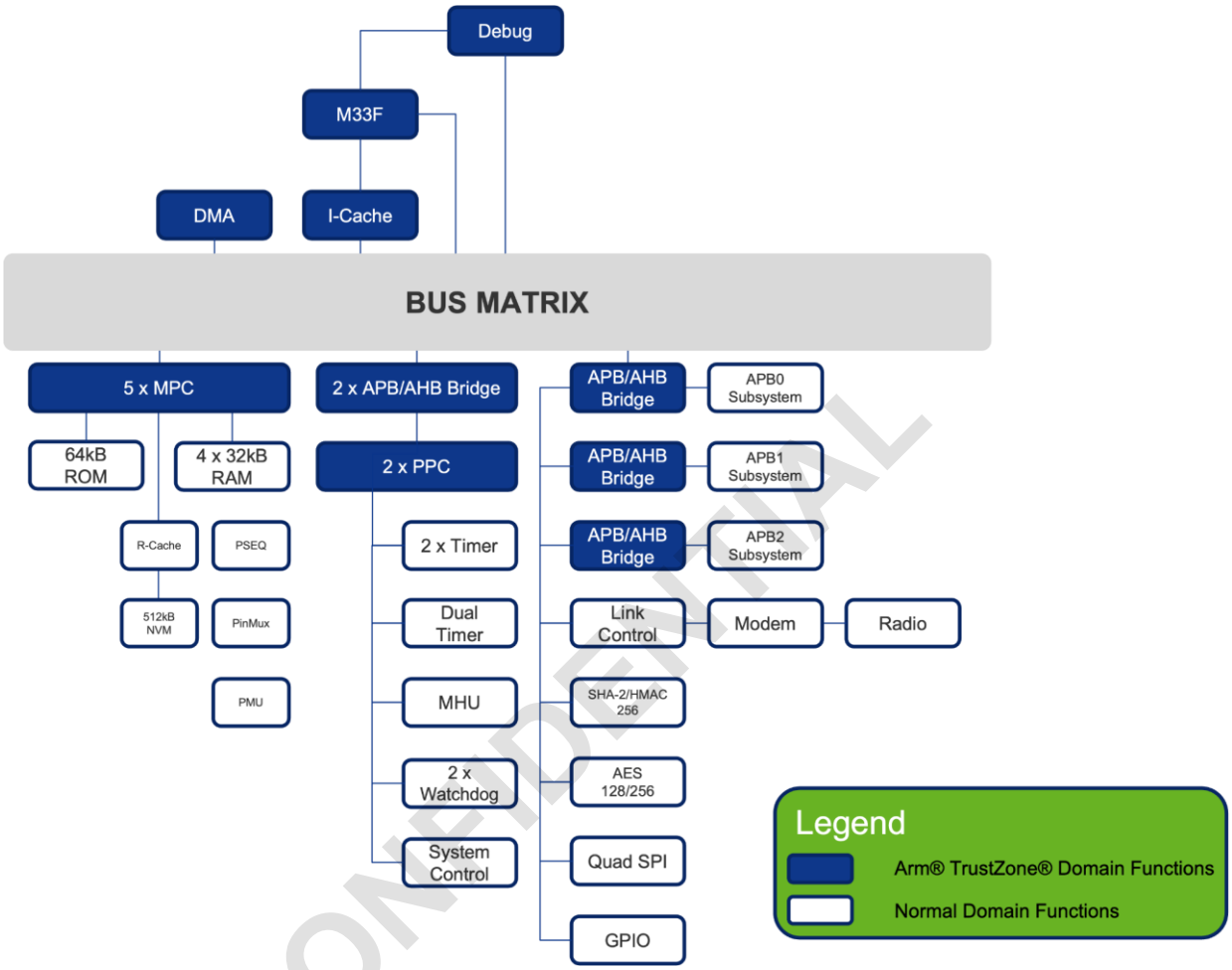


Figure 2-1 Functional Block Diagram

CONFIDENTIAL

2.1 MCU and Memory

The ATM33/e Wireless SoC series contains a 64 MHz 32-bit ARM® Cortex®-M33F processor that is optimized for low-power operation. The processor is a little-endian, 32-bit RISC processor which implements the ARMv8-M architecture specification. It supports all Thumb-1/Thumb-2 instructions. It features four breakpoints, a serial access debug port, 103 interrupts, a single cycle multiplier, full wake-on-interrupt support, and two watch points. The M33F includes an IEEE-754 compliant FPU. The M33F also supports ARMv8-M DSP extension for audio and sensor applications. The M33F core is configured with a 4 KB instruction cache to minimize instruction fetch latency.

The on-chip system memory is organized as shown in Table 2-1. Details of the registers are documented in the Register Specification available on the Atmosic Support website.

Start	Stop	Block
0x0000 0000	0x0008 FFFF	ROM + NVM [non-secure]
0x1000 0000	0x1008 FFFF	ROM + NVM [secure]
0x2000 0000	0x2001 FFFF	SRAM [non-secure]
0x3000 0000	0x3001 FFFF	SRAM [secure]
0x4000 0000	0x4030 4FFF	TImers, SLow 32k timer, Watchdog, GPIO, UART, PWM, SPI, KSM, QSPI/FLASH, I2C, NVM/OTP, PMU, PSEQ, SWD, GADC, Sensor Hub, PDM, I2S, DMA, SHA-2/HMAC 256 and AES-128/256 [Non-secure]
0x5000 0000	0x5030 4FFF	All of the above peripherals aliased in secure space [Secure]

Table 2-1 System Memory Map

CONFIDENTIAL

Non-secure and Secure address aliases are used by the M33F processor in secure and non-secure modes, respectively. The physical mapping of the resource (memory, peripherals) to that alias is managed by the MPC and PPC. Please refer to the Arm® TrustZone Technology for the Arm v8-M Architecture manual.

The ATM33/e includes the following memory components. In general, each core is compliant with ARM AMBA AHB5 or ARM AMBA APB4.

- ROM: 64 KB of ROM. Core elements of the firmware are placed in ROM to add security, as well as extend application space and reduce latency.
- SRAM: 128 KB of SRAM containing both system RAM and data RAM organized as eight 16 KB blocks. The power state of each block in each low power state can be independently controlled.
- NVM: 512 KB of nonvolatile memory to store configuration, calibration data and user application and data, and 64-bits of one-time programmable (OTP) memory to store some PMU and security configurations.
- FLASH: A QSPI master port to interface with an external flash chip if needed (up to 16 MB).

2.1.1 Clocks

These are the primary clock domains on the ATM33/e: core clock, backplane clock, and a variety of fixed frequency clocks.

- Core clock is used as a system real time clock, and is used to sequence transitions between low power and active states. The core clock can be derived from either an external 32.768 kHz crystal, or from an on-chip RC oscillator based circuit. For applications that use the on-chip RC oscillator, the following items need to be carefully considered:

    1) Temperature:
       The default settings are valid for an operating ambient temperature range of -20°C to 85°C.

    2) Current Consumption:
       The default settings will increase the average current during a BLE activity by 2-3 µA, though depending on temperature there will be an additional increase in current due to faster drift.

    3) Timing Accuracy:
       Applications that require tracking real time accurately need to account for RCOSC frequency variability

- The RF clock consists of an 8 MHz and a 16 MHz fixed frequency clocks used by the link controller, the modem, and the radio subsystems. These clocks must be derived from the external 16 MHz crystal oscillator.
- The backplane clock can vary between 0.5 to 64 MHz, and is used by the MCU and some peripherals. The backplane clock can be derived from either a crystal based circuit, or the on-chip RC oscillator circuit.
- The PDM clock is either a 12 or 16 MHz fixed frequency clock.
- The I2S clock is a 16, 24, or 32 MHz fixed frequency clock.
- The majority of the cores are clocked by the backplane clock but there are these exceptions:
    - link controller, modem: 8 MHz and 16 MHz only
    - PWM: 16 MHz only; independent of backplane clock frequency
    - UART0,1: 16 MHz only; independent of backplane clock frequency
    - GADC: 16 MHz only; independent of backplane clock frequency

CONFIDENTIAL

- TRNG: 16 MHz only; independent of backplane clock frequency
- TIMER0,1: 16 MHz only; independent of backplane clock frequency
- DUAL_TIMER: 16 MHz only; independent of backplane clock frequency

RC Oscillator Measurement

The rcosc_cal core provides a way to find the frequency of an imprecise clock (RC oscillator). It takes 2 clocks, one at 32 kHz (slow) and the other at 16 MHz (fast), as input and runs two separate counters on each clock for a duration. Of these two clocks, one is the reference and the other imprecise. At the end of the run, the user finds the unknown frequency by the equation : F_unknown = F_ref * Count_unknown/Count_ref. The variables represent the following quantities:

1) F_unknown: unknown frequency of the imprecise clock.
2) F_ref: frequency of the reference clock.
3) Count_unknown: the value of the counter that runs on the imprecise clock.
4) Count_ref: the value of the counter that runs on the reference clock.

Measurement can be manually activated by the software if **cal_ctrl**.*hw_cal_*sel[1] is set to 0 or automatically triggered by the hardware. In the software-controlled mode (**cal_ctrl**.*hw_cal_sel* = 0), the user can set the duration of the run in terms of the slow clock cycles in **slow_clk_set** register and trigger the counting by setting **cal_ctrl**.*sw_cal* to one. The ongoing status is indicated by stat.running. Once the counting is completed, the values of the 2 counters are available in **slow_clk_cnt** and **fast_clk_cnt** registers.

One use case is to find the frequency of the 32 kHz generated by the RC oscillator. The user then must program to have the 16 MHz clock input to the core be sourced by the XTAL clock. In this case, the variables in the equation are:

1) F_unknown: frequency of the 32 kHz RC clock.
2) F_ref: frequency of the XTAL 16 MHz.
3) Count_unknown: slow_clk_cnt because the 32 kHz (slow) clock is the imprecise clock.
4) Count_ref: **fast_clk_cnt**

Another use case is to find the frequency of the 16 MHz generated by the RC oscillator. The user then must program to have the 16 MHz clock input to the core be sourced by the RC oscillator. Now, the reference clock is the slow clock. If the slow clock also comes

[1] Notation convention: **register name**.*bit name*, for example, **cal_ ctrl**.*hw_cal_sel* is register cal_ctrl, bit hw_cal_sel.

CONFIDENTIAL

from an RC oscillator, then the user must find the frequency of the slow clock first. Once the slow clock is known, the variables in the equation are as follows:

1) F_unknown: frequency of the 16 MHz RC clock.
2) F_ref: frequency of the slow clock.
3) Count_unknown: fast_clk_cnt because the fast clock is the imprecise clock.
4) Count_ref: slow_clk_cnt.

In the hardware-controlled mode (**cal_ctrl**.*hw_cal_sel* = 0), each core's run is enabled when the signal "rif_rf_busy" is high, which implies that the source of the 16 MHz clock to the core is XTAL. This mode is strictly used to calculate the frequency of the slow clock, assuming the source of the 32 kHz is RC oscillator. In this mode, the software only needs to read the values of the counters and apply the equation to find the frequency of the slow clock. The duration of the counting is dictated by the "rif_rf_busy", unlike the software-controlled mode in which the counting duration is set by **slow_clk_set**.

WRPR

The WRPR core contains registers for clock gating, for reset forcing, and for configuration settings of cores that are not directly CPU accessible. Clock gating and reset forcing functionality is limited to the APB cores located on the APB bus. By default each downstream core is in reset and is clock gated off. To enable a downstream core, the recommended approach is to enable clock and then disable reset in separate APB writes.

The core contains the PINMUX configuration registers (**psel_A** through **psel_I**, **pupd_ovr/pupd_ovr_val**). This is where the assignment between physical pin and functional core is made. Each physical pin can be configured independently. Not all SoC functions are mapped to all physical pins.

The core contains the interrupt masks. This is where the user can configure which collection of interrupt sources are combined into each interrupt. The core also contains both the chip ID field and the chip revision field.

CONFIDENTIAL

2.1.2 APB Subsystem

The APB0 subsystem has 16 APB slave blocks. It contains a bridge to and from the AHB bus and concatenates the interrupts of the underlying APB slaves. The cores spi0 and spi1 are meant to communicate with an off-chip SPI slave device. The core spi2 is one of three possible masters of the quad SPI interface. See Table 2-2.

The APB1 system has 13 APB slave cores. It contains a bridge to and from the AHB bus and concatenates the interrupts of the underlying APB slaves. See Table 2-2.

- The rif core is external. Only its APB signals are decoded in the APB1 subsystem.
- The radio core serves as the bridge to the radio core.
- The mdm core is external. Only its APB signals are decoded in the APB1 subsystem.
- The pmu core serves as the bridge to the PMU core.
- The pseq core is external. Only its APB signals are decoded in the APB1 subsystem.

The APB2 system has 5 APB slave cores. It contains a bridge to and from the AHB bus and concatenates the interrupts of the underlying APB slaves. See Table 2-2.

APB Slave Core	APB0	APB1	APB2
1	timer0	i2c0	pdm0
2	timer1	i2c1	pdm1
3	dualtimer	nvm	wrpr2
4	wrpr0	wrpr1	i2s
5	uart0	rif	clkrstgen
6	uart1	radio	
7	pwm	mdm	
8	spi0	pmu	
9	watchdog	pseq.	
10	oled	gadc	
11	ksm	trng	
12	spi2	rcos_cal - RC oscillator	
13	qdec	shub - Sensor hub	
14	spi1		

CONFIDENTIAL

APB Slave Core	APB0	APB1	APB2
15	slowtimer		
16	qspi		

Table 2-2 List of APB Slave Cores

2.1.3 Reset

The PMU releases the chip-wide reset once the power supplies have stabilized. There is no explicit reset pin on the ATM33/e but the user can use PWD for an equivalent purpose. Many of the internal modules can additionally be reset through a software register write. After reset, the default state of most GPIOs will be in high-z state, except for P0/P1 which default to inputs (for SWD).

2.1.4 Wakeup Timer

Wakeup timer is a 40-bit timer based on the 32 kHz clock. When this timer is enabled during SoC Off mode, it will determine the SoC Off duration.

2.1.5 General Purpose Timers

There are four general purpose timer cores: Timer0, Timer1, Dual Timer, and Slow Timer. The value of the timers are readable by the CPU. All timers are clocked by the 16MHz fixed clock except for the Slow Timer which uses the fixed 32 kHz clock. All these timers stop when the system enters a low power mode. The state of the timer is maintained during retain mode but is reset during hibernate mode.

- Timer0 and Timer1, if enabled, will decrement from a 32 bit reload value to 0 triggering a maskable interrupt as it transitions from 1 to 0. The reload value is loaded as the next timer value when the timer reaches 0.
- Dual Timer on APB bus contains two timers and each timer is independent of the other but sharing a single interrupt output. Each timer counts down and can run in one-shot, periodic, or free running mode. The timers are configurable to be either 16 bit or 32 bit. Additionally there is a rescale in front of each timer that can reduce the incoming clock by 1x (no change), 16x, or 256x.

CONFIDENTIAL

- Slow Timer on APB bus is a 40-bit count down timer with three programmable thresholds. The interrupt is optionally asserted when the counter counts from one above the threshold to the threshold. It is recommended that the timer be read twice and the value used only if the values from both reads are the same. If the values are not the same then the read should happen a third time. The timer is updating approximately every 488 cycles of the 16 MHz clock.

2.1.6 Interrupts

The SoC supports 10 processor exception interrupts and 128 maskable interrupts. The maskable interrupts are listed in Table 2-3.

CONFIDENTIAL

#	Name	#	Name	#	Name	#	Name
0	NSECUREWDOG_RESETREQ_IRQn	32	UARTRX0_IRQn	64	MDM_IRQn	96	PORT0_0_IRQn
1	NSECUREWDOG_INT_IRQn	33	UARTTX0_IRQn	65	reserved	97	PORT0_1_IRQn
2	S32K_TIMER_IRQn	34	UARTRX1_IRQn	66	reserved	98	PORT0_2_IRQn
3	TIMER0_IRQn	35	UARTTX1_IRQn	67	reserved	99	PORT0_3_IRQn
4	TIMER1_IRQn	36	SPI1_IRQn	68	DMA0_IRQn	100	PORT0_4_IRQn
5	DUALTIMER_IRQn	37	reserved	69	DMA1_IRQn	101	PORT0_5_IRQn
6	MHU0_CPU0_IRQn	38	reserved	70	DMA2_IRQn	102	PORT0_6_IRQn
7	MHU1_CPU0_IRQn	39	reserved	71	DMA3_IRQn	103	PORT0_7_IRQn
8	reserved	40	SPI0_IRQn	72	BLE_SW_IRQn	104	PORT0_8_IRQn
9	MPC_COMB_SEC_IRQn	41	UART0_OVERFLOW_IRQn	73	BLE_FINETGT_IRQn	105	PORT0_9_IRQn
10	PPC_COMB_SEC_IRQn	42	UART1_OVERFLOW_IRQn	74	BLE_TIMESTAMP_TGT1_IRQn	106	PORT0_10_IRQn
11	MSC_COMB_SEC_IRQn	43	KSM_IRQn	75	BLE_TIMESTAMP_TGT2_IRQn	107	PORT0_11_IRQn
12	BRD_ERR_COMB_IRQn	44	SLWTIMER_IRQn	76	BLE_TIMESTAMP_TGT3_IRQn	108	PORT0_12_IRQn
13	ICACHE_IRQn	45	SPI2_IRQn	77	BLE_CRYPT_IRQn	109	PORT0_13_IRQn
14	reserved	46	PWM_IRQn	78	BLE_SLP_IRQn	110	PORT0_14_IRQn
15	SYS_PPU_IRQn	47	reserved	79	BLE_HSLOT_IRQn	111	PORT0_15_IRQn
16	CPU0_PPU_IRQn	48	SPI_RADIO_IRQn	80	BLE_FIFO_IRQn	112	PORT1_0_IRQn

CONFIDENTIAL

#	Name	#	Name	#	Name	#	Name
17	CPU1_PPU_IRQn	49	SPI_PMU_IRQn	81	BLE_ERROR_IRQn	113	PORT1_1_IRQn
18	CPU0DBG_PPU_IRQn	50	I2C0_IRQn	82	BLE_HOP_IRQn	114	PORT1_2_IRQn
19	CPU1DBG_PPU_IRQn	51	I2C1_IRQn	83	BLE_ISOTS_0_IRQn	115	PORT1_3_IRQn
20	CRYPT_PPU_IRQn	52	NVM_IRQn	84	BLE_ISOTS_1_IRQn	116	PORT1_4_IRQn
21	reserved	53	GADC_IRQn	85	BLE_ISOTS_2_IRQn	117	PORT1_5_IRQn
22	RAM0_PPU_IRQn	54	TRNG_IRQn	86	BLE_ISOTS_3_IRQn	118	PORT1_6_IRQn
23	RAM1_PPU_IRQn	55	RCOS_IRQn	87	BLE_ISOTS_4_IRQn	119	PORT1_7_IRQn
24	RAM2_PPU_IRQn	56	PDM0_IRQn	88	BLE_ISOTS_5_IRQn	120	PORT1_8_IRQn
25	RAM3_PPU_IRQn	57	PDM1_IRQn	89	BLE_ISOTS_6_IRQn	121	PORT1_9_IRQn
26	DBG_PPU_IRQn	58	PDM2_IRQn	90	BLE_ISOTS_7_IRQn	122	PORT1_10_IRQn
27	reserved	59	PDM3_IRQn	91	PSEQ_IRQn	123	PORT1_11_IRQn
28	reserved	60	I2S_IRQn	92	PMU_IRQn	124	PORT1_12_IRQn
29	reserved	61	reserved	93	PORT0_ALL_IRQn	125	PORT1_13_IRQn
30	reserved	62	reserved	94	PORT1_ALL_IRQn	126	PORT1_14_IRQn
31	reserved	63	reserved	95	reserved	127	PORT1_15_IRQn

Table 2-3 List of Maskable Interrupts

CONFIDENTIAL

2.1.7 Power Modes

The SoC supports five primary power states which are Active, Retention, Hibernation, SoC Off and Powerdown (PWD). Each primary state may have several secondary states depending on the number of active power domains and clock gating. Refer to the **Understanding Low Power Modes for ATM33/e Series White Paper** for additional details.

1) **Active**: All regions of the SoC are powered on. Active power can be optimized by utilizing clock gating registers and/or by putting the CPU into Wait for Interrupt (WFI).
    a) CPU Idle: When the CPU function is not required, it can be placed into WFI state to conserve power.
    b) Bluetooth LE Deep Sleep: Bluetooth subsystem is powered down while the remainder of the ATM33/eis powered up. This state is useful when data needs to be processed but does not need to be transmitted over RF.

2) **Retention**: All or some of the 128 kB SRAM, in increments of 16 kB, can be retained. All register/flip-flop states are retained. Digital I/O's will hold the state they were at when the transition into either Hibernate or Retain started. Wake can be from a timer expiring, activity detected on GPIO's, activity detected on the keyboard, activity detected on the mouse, the sensor hub reading measurements crossing a threshold, and the detection of a connection over the SWD interface. All selections about how to wake need to be programmed before the transition into the low power state is triggered. The SRAM supply voltage can be lowered to further reduce leakage power consumption.

3) **Hibernation:** Powers down system memory. Retains only a minimal amount of flip-flop state. Retains I/O state. Wake up setting must be programmed before transitioning into this state.

4) **SoC Off:** all digital domains including the top level digital domain are powered down, but the PMU remains on in an ultra low-power state with limited functionality. The system must do a complete, cold start reboot when returning from this state to an active state. Wake mechanisms are limited to (a) special 40-bit timer, (b) external pin edge on P5, (c) ultra-low power analog comparator with input on either P3 or P4.

    a) To wake on timer, the desired powered down duration in 32.768 kHz cycles is written into the 40-bit value in PMU_TOP

CONFIDENTIAL

pmu1.*timer_target_msb* and **pmu0**.*timer_target_lsb*. A wake event is initiated when this timer crosses from 1 to 0. Additionally **pmu1**.*how2wkup*[2] needs to be set to enable wake on timer.

b) To wake on GPIO activity, **pmu1**.*how2wkup*[0] needs to be set to denote wake on GPIO activity. The wakeup circuitry will check for a high level on P5. This functionality is only valid with an external I/O supply.

c) To wake on threshold crossing, the user sets **pmu1**.*how2wkup*[1] and monitors either P3 or P4 via **pmu1**.*lpcomp_in_sel* with the reference voltage adjusted via **pmu11**.*lpcomp_ref*. This functionality is only valid with an external I/O supply.

5) **Powerdown** (PWD pin asserted): All power domains including the PMU are completely shut off. No supplies are internally generated or maintained. This state is only valid with an external supply on VBAT. All GPIOs are held in high-z state in this state.

CONFIDENTIAL

2.1.8 OTP Memory

There are 64 bits one-time programmable (OTP) memory bits, defined as follows:

Function Name	Bit Locations	Description
disable_xtal32k	19	If set, the external 32k crystal oscillator is not used.
disable_vddio_gen	21	If set, disables internal VDDIO generation. See Table 2-14 for details.
disable_vdd_gen	22	If set, disables internal AVDD1 and DVDD1 generation
disable_vbatli	23	If set, disables VBATLI functionality.
vbat_level	24	Set the battery voltage level. See Table 2-14 for details.
batt_type	26:25	Set the battery type. See Table 2-14 for details.
vbat_good	29:27	Sets the High-to-Low threshold for VBAT: 0.9 + vbat_good * 0.025 (0.9 - 1.075 range) VBATLI: 1.2 + vbat_good * 0.05 (1.2 - 1.55 range)
vbat_brownout	36:30	Sets the control voltage for brownout. See Table 2-14 for details.
mppt_type	38:37	Sets the MPPT type. See Table 2-14 for details.
vstore_max	46:44	Max voltage regulated at VSTORE.

Function Name	Bit Locations	Description
		See Table 2-14 for details.
rram write locks	55:48	Locks down various sections in RRAM. See Table 2-4 for details.
sec_dbg_config	61:60	SWD Secure Debug Configuration. See Table 2-5 for details
rram_jtag_bypass	62	If set, disables RRAM side-channel access using JTAG BIST
reserved	all other bits	Reserved bits. Do Not Use.

To read the OTP bits, it is recommended to use either MPTool, or the APIs available in the atm_otp driver in software. To do it manually, do the following:

1) Enable the OTP AHB clock: CMSDK_WRPR1->APB2_CTRL = 1
2) CMSDK_NVM->OPMODE = 0
3) NVM_OPMODE__READ__SET(CMSDK_NVM->OPMODE, 1)
4) NVM_OPMODE__GO__SET(CMSDK_NVM->OPMODE, 1)
5) The OTP bits will be saved in CMSDK_NVM->DATA0 (bits 0-31) or CMSDK_NVM->DATA1 (bits 32-63)
6) Disable the OTP AHB clock: CMSDK_WRPR1->APB2_CTRL = 0

To write the OTP bits, it is recommended to use MPTool to have these bits set during the manufacturing flow. To program bit <BIT> (between 0-63):

1) Configure VAUX to 2.5V setting:
2) Enable the OTP AHB clock: CMSDK_WRPR1->APB2_CTRL = 1
3) CMSDK_NVM->CTRL = 0
4) NVM_CTRL__NUM_PGM_BIT__MODIFY(CMSDK_NVM->CTRL, <BIT>)
5) CMSDK_NVM->OPMODE = 0
6) NVM_OPMODE__PROGRAM__SET(CMSDK_NVM->OPMODE, 1)
7) NVM_OPMODE__GO__SET(CMSDK_NVM->OPMODE, 1)
8) Wait until CMSDK_NVM->STATUS goes to zero

CONFIDENTIAL

2.1.9 Debug Access

ATM33/e supports Serial Wire Debug (SWD) interface which provides access to the ARM debug port.

The SWD interface is compliant with the ARM Debug Interface Specification v5. There is a single Serial Wire Debug Port (SW-DP), 1 single APB Access port (APB-AP) and a single AHB Access Port (AHB-AP). The AHB-AP connects to the AHB backplane and has access to the entire memory mapped space. The debugger and the Cortex®-M33F share the same internal AHB master port. To prevent execution of erroneous software that blocks debug access, the Cortex®-M33F should be instructed to stay in an idle state by setting the pin P25 high and then rebooting the ATM33/e. If either CSYSPWRUPREQ or CDBGPWRUPREQ in the Serial Wire Debug Port (SW-DP) register CTRL/STAT is set, the SoC will return from a lower power mode (if in a low power mode when the bits are set) and the SoC will not go into subsequent low power modes. Refer to the **ARM Debug Interface Specification v5** for more information.

When debug is not needed, SWD or JTAG or trace debug pins can be used for functional purposes.

2.1.10 Trace Port Interface Unit

The ATM33/e debug system includes the ARM's Coresight Embedded Trace Macrocells (ETM trace unit). In ATM33/e, we assign 4 signals for ETM data lines. See `tracedata_iq0`, `tracedata_iq1`, `tracedata_iq2`, `tracedata_iq3` in Pin Multiplexing. For the traceclk, an 8 MHz clock, an indirect way is required to generate this clock. The user can program a PWM core to continuously output a 8 MHz signal.

2.1.11 Boot Flow

The high level boot flow and timing is shown in Figure 2-2, for the cases where Secure MCUBoot is not used. There is around 12 ms of boot where the PMU brings up various items, such as the 32 kHz crystal oscillator and voltage rails, until the software application boots. Timings from then depend on the specific software loaded.

In the middle of the boot, the software will check P25, and if it's high it will not boot the software loaded, and go into an idle state waiting-for-interrupt (WFI). This mode is used during manufacturing to load a software image onto the NVM or program OTP bits.

CONFIDENTIAL

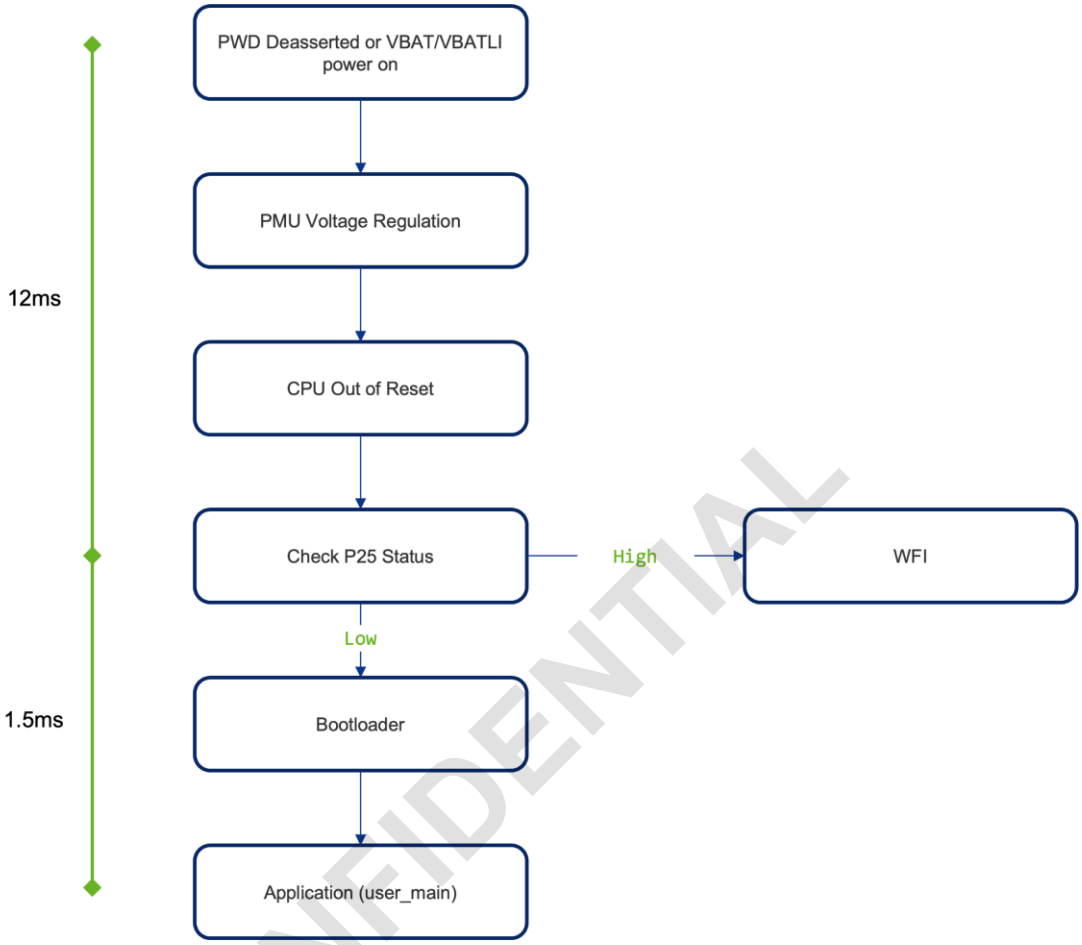


Figure 2-2 Boot Flow from PWD Deasserted/VBAT Power on to SW bootloader

More details of the PMU voltage regulation sequence can be found in the PMU section.

CONFIDENTIAL

2.2 Security

The ATM33/e is designed to address IoT security requirements against a variety of threats. Securing an IoT device starts with hardware that provides a silicon-based Root of Trust (RoT) and extending that trust to the software execution environment that runs trusted firmware. The security architecture of ATM33/e follows industry standard best practices against spoofing, tampering, repudiation, information disclosure, and elevation of privileges attacks. These practices are embodied in a set of security goals that secure platforms should aspire to. These goals are described by the Platform Security Architecture industry consortium (see: https://www.psacertified.org/).

2.2.1 Security Goals

The ATM33/e has been designed with hardware features that address an industry-standard security goal or best practice. These are enumerated below.

Unique ID

Secure devices should be uniquely identifiable and this identity should be immutable and attestable. A unique ID can be stored in the Secure Journal, an append-only immutable storage facility in RRAM. Alternatively a unique shared private key can be stored in the AES/HMAC side-loaded Private Key Storage blocks.

Attestation

Attestation is a process that verifies the identity and state of a device by a trusted party. There are many attestation protocols involving both symmetric and asymmetric cryptography. ATM33/e provides building blocks to build a trusted attestation mechanism that can interact with a trusted verifier. An implementation can place a per-device unique private key in Private Key Storage. These keys are only readable by the crypto hardware engines (AES or HMAC) and invisible to software. The side-loaded encrypt/decrypt operations can be employed in an attestation protocol.

Secure Boot

Secure boot ensures that only authorized firmware can be executed on the device. Secure boot starts with the ROM which redirects the boot chain to a specific RRAM location hosting a bootloader image. Although the ROM does not authorize the firmware found there, it can only be directed to jump to that fixed location. The fixed location in RRAM however resides in a Secure RRAM block which can be made immutable and designed to extend boot functionality. The ATM33/e SDK provides a secure boot reference implementation using MCUBoot. This implementation uses ECDSA (Elliptical Curve Digital Signature Algorithm) to verify a signed firmware image. The platform provides a SHA256 engine to accelerate image hashing operations, a key function of ECDSA. Once the image is verified the secure boot loader transfers control to the verified image for execution. This is also a fixed location in RRAM known to the bootloader. Prior to handing control to the image, the secure bootloader can also configure RRAM/ROM protection controls to optionally read-protect boot ROM and the secure boot loader itself.

CONFIDENTIAL

Secure Update

All image updates delivered over the air or programmed via wire must be signed and will be verified by the secure boot process prior to execution. Verification prior to installation may be performed but all images are verified before execution.

Anti-Rollback

A secure firmware update should prevent downgrades to firmware versions that are less secure or contain known exploitable security bugs. An authorized rollback is possible as long as the firmware does not downgrade the security level of the device.

To enhance anti-rollback capabilities, the platform provides a set of secure counters for this purpose. These are nonvolatile, increment-only counters that saturate at the wear level endurance of the underlying RRAM cells.

Isolation and Interaction

A system with hardware enforced isolation prevents compromised software from compromising secure or trusted software. This is accomplished by isolating secure services in a separate processing domain away from potentially exploitable (non-secure) software services. Isolation however does not prevent interaction between the two domains. An application in the non-secure domain can make requests to the secure domain through an interaction interface. The secure domain can maintain local secrecy and privacy for such requests.

ATM33/e provides isolation and the means of interaction through ARM Trustzone-M technology.

Secure Lifecycle

The security lifecycle represents the authorized interactions of the device as it moves from development, assembly/test, platform provisioning, deployment, operation, debugging (RMA) and in some cases device termination/retirement. Some lifecycle states are irreversible, for example platform provisioning in the factory to load private keys and unique identifiers. The irreversibility of state is facilitated through the Secure RRAM and Secure Journal. Debug access must be controlled to maintain the security of the device as it moves through its lifecycle. Initially at manufacturing full debug access is enabled to install images, load data, configure settings and finally seal the product. Sealing the product requires disabling the debug port from future interactions, however some devices may require debug port access for RMA or product decommissioning purposes. ATM33/e provides fully disabled and authenticated debug modes using hardware enforced Secure Debug.

CONFIDENTIAL

2.2.2 Atmosic Security Blocks

The ATM33/e SoC employs additional security blocks to complement ARM TrustZone-M technology. Trustzone-M cannot secure a platform on its own, nor can it provide a silicon root of trust without the help of additional hardware support.

AES-128 & AES-256

The ATM33/e has an AES (FIPS-197) hardware accelerator that supports 128-bit and 256-bit key lengths. The keys can be provided to the key manager directly by software or they can be loaded through a hardware assist sideload mechanism from a key kept safely in a private key storage area.

A variety of cipher modes are supported: ECB, CBC, CFB, OFB, and CTR. To access the AES block software can enable clocks and remove the reset by clearing WRPR2.APB7_CTRL.sreset and setting WRPR2.APB7_CTRL.clk_enable. The shared AES key is stored in the AES.KEY_SHARE0_x and AES.KEY_SHARE1_x registers (up to 256 bits of key material). The AES engine can be started with a control word written twice to AES.CTRL_SHADOWED. The control word sets the direction (encrypt/decrypt), the mode (ECB, CBC etc..), the key size (128 or 256) and manual or auto start. Auto start will start encryption as soon as data is available. In manual start mode software can load data and later manually use AES.TRIGGER.START to start the operation. Software must check the status register AES.STATUS for the INPUT_READY bit before loading data. The plaintext block (16 bytes) are input through AES.DATA_IN_x registers. The ciphertext block is read from AES.DATA_OUT_x registers. Other cipher modes that require an IV can program the IV through AES.IV_x registers.

To side load a key , instead of loading AES.KEY_SHARE0/1 registers, software sets both AES.SIDELOAD_CTRL.sideload_update_en and AES.SIDELOAD_CTRL.sideload_val together and then clears AES.SIDELOAD_CTRL.sideload_val. This sets up the internal sideload operation. Software then reads the key (16 or 32 bytes) to complete the sideload. This key can only be read from the private key storage area. Once read, the AES.SIDELOAD_CTRL.sideload_val must be set to 1 to indicate the key is now valid. The control word for AES.CTRL must set the sideload bit to use the internal sideload key buffer (not visible to software) instead of the KEY_SHARE registers. Data input and output are handled in the same way as the non-sideloaded key method. To protect the privacy of the sideloaded key an OTP option can hide the secure key area.

CONFIDENTIAL

SHA-256 & HMAC-256

The ATM33/e has a SHA-256/HMAC-256 data hashing hardware engine. The engine can compute a non-secure hash for applications such as firmware image hashing in accordance with ECDSA in Secure Boot. The engine also supports secure HMAC-256 data hashing (RFC2104) with software programmed or hardware side-loaded private keys.

To use the engine, software must enable the SHA clock via CLKRSTGEN.USER_CLK_GATE_CONTROL.sha_hclk. The register SHA2.CONTROL.sha_en configures the mode for SHA256 while SHA2.CONTROL.hmac_en enables HMAC-256 mode. The engine provides byte-swap conversion (endian conversion) for data written to the 32-bit data FIFO ports as well as the digest result loaded into the digest registers: SHA2.DIGESTx. The SHA2.CONTROL.byte_swizzle and SHA2.CONTROL.digest_swizzle are used to enable byte swapping. If the engine is used in HMAC mode, the key can be loaded by software into SHA2.KEYx registers. If the key is sideloaded from the Private Key Storage area, software must toggle SHA2.SIDELOAD_CTRL.sideload_ptr_clear to 1 and back to 0. Software can then set SHA2.SIDELOAD_CTRL.sideload_update_en to 1 to enable the side load mechanism. To sideload a key, software reads the private key sequentially from RRAM, 32-bits at a time. The OTP setting to lock the private key storage is not required to sideload the keys. None of the key material is loaded into any SHA2.KEYx register. If the private key area is locked via OTP, the sideloaded read operation always returns 0 (the key is hidden), the read triggers the key data to sideload into the HMAC engine. At no point can this key data be read out as it is not loaded into any CPU visible key register.

Once keys are programmed and the engine configured, the hash operation can be started by setting SHA2.CMD.hash_start to 1. The engine will be ready to receive data through its SHA2.FIFO_PUSH register (bytes or words at a time). The FIFO is 16-words (64 bytes) deep. Software can load up to 64 bytes at a time and poll SHA2.STATUS.fifo_empty to load the next 64 byte block. After moving all data through the FIFOs the digest can be computed by setting SHA2.CMD.hash_process to 1. This indicates to the engine that all data has been transferred and the final digest can be computed. Software must poll SHA2.INTERRUPT_STATUS.hmac_done before reading the digest result from SHA2.DIGESTx registers. The SHA engine also provides interrupt support for FIFO loading and status monitoring via SHA2.INTERRUPT_MASK and SHA2.INTERRUPT_STATUS registers.

CONFIDENTIAL

When HMAC is used, software can wipe any trace of the secret key used in the engine by writing (any data) to SHA2.WIPE_V. For sideloaded keys this wipe operation is not required as keys are never loaded directly to registers.

TRNG (True Random Number Generator)

The ATM33/e Series has a True Random Number Generator (TRNG) which generates a single 32-bit random number per invocation. The number is generated when the radio is first powered up or when requested by software. Arbitrarily long random numbers can be achieved by repeatedly invoking this random number generator. The outputs of TRNG can be used as seeds to a deterministic or pseudo random number generator (DRNG/PRNG) to improve entropy. Random numbers of this type are better suited for cryptography. Some generators use a cryptographic algorithm (CHACHA20, SHA, AES) to permute a stream of TRNG samples into a random number stream. The random number stream is suitable for public and private keys used in various key agreement and signing protocols. The random number generation completes in as short as 64 µsec (1,024 clk_mpc clock cycles). In the unlikely event that the random number is not generated within 2 msec (32,768 clk_mpc clock cycles), an interrupt is generated to indicate the TRNG failure. In general, random number generation should be scheduled to run during periods of inactivity.

To access the TRNG, software can enable register access and take the core out of reset via WRPR1.APB11_CTRL. Software can enable TRNG ready interrupts by setting TRNG.INTERRUPT_MASK.trng_ready and clear the interrupt via TRNG.RESET_INTERRUPT. The TRNG requires the radio to be powered on in order to perform the sample collection. Software can enable auto sample collection whenever the radio comes out of low power mode using the option TRNG.CONTROL.launch_on_radio_up. After a sample is collected an interrupt will be generated. Please consult the SDK on TRNG configuration and sample collection. On a TRNG interrupt, software can read the TRNG.TRNG register to obtain the captured sample.

Secure Data Area

ATM33/e provides secure non-volatile storage with security related properties such as immutability and privacy. These mechanisms help establish a silicon root of trust. These features are controlled using one-time programmable (OTP) settings that are programmed in the final steps of manufacturing. Refer to OTP memory section for details. The following OTP table identifies which OTP configuration bits configure the property of a secure data structure or block in RRAM.

CONFIDENTIAL

OTP Bit	Start	Stop	Size	Block	Effect When Set
54	0x0008 FF00	0x0008 FFFF	256	Private Key Storage	Write and read protected block
53	0x0008 FEF0	0x0008 FEFF	16	Secure Counters (8x16b)	Enables secure journal and secure counters
–	0x0008 F800	0x0008 FEEF	1776	Secure Journal	See OTP53
52	0x0008 F000	0x0008 F7FF	2048	Protected Data	Write protects block
51	0x0001 C000	0x0001 FFFF	16384	Boot Block 3	Write protects block
50	0x0001 8000	0x0001 BFFF	16384	Boot Block 2	Write protects block
49	0x0001 4000	0x0001 7FFF	16384	Boot Block 1	Write protects block
48	0x0001 0000	0x0001 3FFF	16384	Boot Block 0	Write protects block

Table 2-4 Secure RRAM Data Structures

CONFIDENTIAL

Private Key Storage

The private key storage area can hold AES and HMAC private keys that can be sideloaded into either AES or HMAC crypto engines. An OTP option can permanently hide the keys but does not disable sideloading. Sideloading for either AES/HMAC can be tested with fully visible keys in the key area during development and later locked down (via OTP) once the product is sealed. The 256 bytes of storage is divided into eight 32-byte (256-bit) key slots. The HMAC-256 can sideload and use any of the eight keys. The AES-256/128 engine however requires that the side loaded key matches the AES key size operation (256 or 128). Of the eight key slots, key slot 0..3 can only contain a 256-bit key while slots 4..7 can contain only 128-bit keys. The AES engine checks which slot was sideloaded and will generate an error if the key slot does not match the software programmed key size to encrypt or decrypt. For example sideloading an AES-256 key slot and subsequently using it in an AES128 operation is not permitted.

Secure Counters

This region of RRAM represents eight 16-bit secure non-volatile counters. Until the OTP value is programmed this region behaves and operates as ordinary RRAM. When the OTP value is programmed the RRAM location now operates as Secure Counters. Secure counter 0 is reserved for usage in the Secure Journal. Note that when the secure journal is activated (generally the case in order to protect ATE and calibration data) the secure counters are also activated.

Secure Journal

This region of RRAM represents a data storage area to store system parameters and configurations traditionally kept in less-area efficient one-time programmable memory (OTP). By default the region operates as ordinary RRAM for data storage. However when the secure counters are activated (via OTP), the region can become an append-only journal where previous writes can be locked (immutable) in place by managing a secure counter (counter 0 is assigned to the journal).

Protected Data

This area reserves 2KB of data that can be write protected for the lifetime of the device. This region may be used to store user defined data or patch code. Patch code storage is intended for a future patchable secure boot ROM.

Boot Blocks

This area of RRAM is used to store a secure bootloader. In practice the secure bootloader boot blocks should be locked as the ROM currently does not perform image verification of the code located at the beginning of boot block 0. Each boot block is 16KB and only the blocks that comprise the secure bootloader require locking. The maximum size of a secure bootloader is 64KB. An unlocked boot block can be used as ordinary RRAM for code or data storage.

RRAM/ROM Protections

The ATM33/e has additional layers of protection for non-volatile memories that store data and code. Although some protections are provided by TrustZone-M memory protection controllers (MPCs), these protections can be circumvented if an attacker is able to gain elevated privileges to execute secure mode code. To prevent such an exploit, the RRAM and ROM subsystems provide additional registers that implement “sticky” protection bits to write protect (read-only) to RRAM memory regions in 2KB increments. “Sticky” protection means that the protection can only be undone by a system reset . A secure bootloader or a secure application at early boot can lock down code areas to prevent

CONFIDENTIAL

modification or even execution regardless of the CPU security state and MPC configuration. Sticky protections are also enforced on all memory observers including DMA and the debug port (provided they are enabled before debug access is allowed).

The sticky protection configuration registers are located in the WRPR0 block. To access these registers you must set CLKRSTGEN.USER_CLK_GATE_CONTROL.sticky_clk to 1. A description of each type of protection and how to configure the protection registers is described in the following sections.

General Write Protection:

All RRAM locations are write protected (locked) on 2KB block boundaries. Write lock settings are volatile and upon reset all RRAM locations are write protected. When software is required to update an RRAM location, the block must be unlocked first. Software can subsequently lock the location after the operation completes. The general write protection controls are located in the registers WRPR0.RRAM_WRITE_PROTECTION[0..7], each register bit represents a 2KB block. Set the bit to 0 to unlock and 1 to lock. The next two sections describe additional "sticky" lock protections that take precedence over general write protection mechanisms.

Sticky Write Protection:

This prevents code or data from being modified in place and subsequently read at run-time. A code modification in RRAM may not result in a reboot where the secure boot loader would be able to catch the image corruption. The system could run with compromised code for some time until a return to the secure boot loader (a reset) halts the system after detecting image corruption.

The sticky write protections for RRAM are configured using the WRPR0.RRAM_STICKY_WRITE_PROTECTION[0..7] registers. Each bit represents a 2 KB block starting at the beginning of RRAM. The eight registers cover the full 512 KB RRAM address space. This write protection takes precedence over the general write protection mechanism.

Secure Counters

ATM33/e provides non-volatile secure counters implemented in a special region of RRAM. There are eight (8) , 16-bit counters located contiguously in this special region. By default the memory locations operate as ordinary RRAM memory until the region is activated via an OTP setting. This option may be set at chip fabrication time in order to

CONFIDENTIAL

enable the secure journal. Once activated, each counter RRAM location can only increment by one when any value is written to it. Reads will return the current value. Secure counters will saturate and stop incrementing at a value of 2047.

Secure Debug

The serial wire debug (SWD) port can be secured with a 2-bit OTP setting (OTP60, OTP61) to operate the SWD port in 3 modes: enabled, disabled and authenticated.

OTP Secure Debug bits	Mode
00	SWD enabled (Default)
01	Disabled
10	Authenticated Debug
11	Disabled

Table 2-5 OTP Debug Settings

The enabled state is the default configuration of the Atmosic device. SWD access is fully granted. The disabled state disallows all SWD access. This state is irreversible and provides the highest level of security. The authenticated state temporarily disables SWD access on power-on and the port continues to be disabled until software (typically a bootloader) can perform an out-of-band authentication challenge (over a communications channel) to verify that the requester is trusted. The software, after confirming trust, can unblock SWD access and normal debugging can proceed. Unlocking access does not persist after a reset and must be unlocked again by software.

Software can write 0 to WRPR0.SECURE_DEBUG_CTRL.soft_disable to unlock the SWD port. If the software determines that no debug authentication is to occur and the system should proceed with normal boot execution, the debug port can be locked down by writing a 1 to WRPR0.SECURE_DEBUG_CTRL.sticky_disable . This will keep the port locked and software will not be able to clear soft_disable to unlock until the system is reset.

Secure DMA

The ATM33/e DMA controller has been enhanced to protect against exploits that could use the DMA controller to bypass the TrustZone protection controllers (MPC/PPC). The DMA controller has 4 channels and each channel can be configured with a nonsecure identity bit. These are respectively cfg_hnonsec (channel 0) and chan_cfg_nonsec for channels 1-3. The identity bit defaults to 1 on reset, meaning that each channel operates as if it was a nonsecure busmaster. Protection mechanisms that trap/block nonsecure

CONFIDENTIAL

access would be fully enforced on the DMA channel. In order for the channel to take on the secure identity, the value must be set to 0. This setting to 0 however is only allowed if the CPU mode is secure. In order to use the DMA to access secure resources the software controlling the DMA channel must also be secure as well.

2.2.3 ARM TrustZone-M

Trustzone-M is an ARM security technology that provides system protection mechanisms that can isolate software to run in secure and non-secure operating environments. Secure and non-secure are operating modes of the processor as well as a property of a system resource (memory, peripherals). The operating environment is also colloquially referred to as secure or nonsecure “worlds”. Trustzone-M allows for safe interactions between these two worlds. Trustzone supports separate operating environments through the addition of CPU/system features focused around execution modes, hardware banking to support separate and isolated states, and resource protection mechanisms. Please refer to ARM’s documentation: “Arm® TrustZone Technology for the Armv8-M Architecture” for more information.

CONFIDENTIAL

Secure and Non-Secure Modes

The ARM TrustZone processor can operate software in a secure or no-secure mode of execution. The default power on reset mode of the system is to operate in secure mode. All resources are configured to only be accessible from the secure mode. Coming out of reset, the system starts in the secure boot ROM environment and continues onward to trusted firmware (secure bootchain). At the end of the secure boot process, the software prepares the nonsecure operating environment and transitions control to that environment.. This involves allocating non secure resources and finally changing the operating mode to nonsecure and jumping to the nonsecure software. Non-secure software is any software that needs to operate outside the trusted secure environment due to its potential vulnerability. Although nonsecure software images are verified through secure boot mechanisms, its runtime operation is still considered vulnerable to exploits such as code injection and buffer overflow attacks. Such attacks are made possible through software interactions with the external world. These attacks pose significant risks that software could be exploited to steal data or compromise the system. A secure environment can isolate critical security functions and data and use secure and authenticated methods to interact with the nonsecure world. In the event the nonsecure

world is compromised, it would not be able to directly access secure data or use functions in an untrusted manner.

Implementation Defined Attribution Unit (IDAU)

A fixed hardware configuration that partitions the address space into secure/non-secure memory aliases. The IDAU also defines addresses that may only reside in the secure space (system and security configuration registers, cache control registers etc..). The default is to mark an address with bit 28 set to 1 to be a secure address. The following table provides an example of this aliasing.

Block or Peripheral	Secure Alias Start	Non Secure Alias Start
RRAM	0x1001_0000	0x0001_0000
UART0	0x5014_4000	0x4014_4000
System Control Block	0x5002_1000	Not defined (IDAU reservation)

Table 2-6 Secure Alias Example

The System Control Block in this example (ARM platform system control registers) can only be accessed through its secure address, the non-secure address is not defined.

The IDAU only defines what addresses are secure (or secure-only) and non-secure and leaves protection mechanisms to the CPU, SAU and other system resource protection controllers (MPCs/PPCs). In the absence of protection controllers both the secure and nonsecure alias can be used to access the same hardware instance, however the CPU will trap any non-secure software attempting to use a secure address alias. To overcome this dual alias problem the SAU can open up select non-secure regions for non-secure access (see SAU section). A more flexible approach is provided using MPCs and PPCs rather than the SAU.

Security Arbitration Unit (SAU)

The SAU is a software configured mechanism to create up to eight regions in the CPU memory map designated as non-secure or ns-callable. Any address that is not covered by a region entry (one of eight possible regions), becomes secured regardless of the IDAU's non-secure definition. The SAU however cannot downgrade an IDAU defined secure address to function as a non-secure address. The SAU can carve out narrow NS-Callable regions to enable non-secure software to call into secure software through a secure gateway.

CONFIDENTIAL

Memory Protection Controllers

Memory protection controllers (MPC) are bus elements that are located in front of a memory block (SRAM, RRAM, FLASH, ROM). The MPC allows subblocks within the memory block to be partitioned between secure and non-secure address spaces or known colloquially as “worlds”. The MPC will filter the access from all bus masters in the system to determine if the security access mode matches the configuration. This avoids the dual alias problem described in the IDAU section where a memory region configured to reside in the secure world should not be accessible via its non-secure alias. Table 2-7 shows the legal combinations of the CPU or DMA security mode using a secure or nonsecure address alias to access a secure or nonsecure memory block. Note that the non-secure CPU mode can never use a secure address alias to access a resource. This violation is automatically caught within the CPU before reaching any MPC.

CPU or DMA Channel Mode	Secure Alias to Secure Block	Secure Alias to Non-secure Block	Nonsecure Alias to Secure Block	Nonsecure Alias to Non-secure Block
Secure	**Allowed**	MPC Fault	MPC Fault	**Allowed**
Non Secure	Secure Fault (CPU) MPC Fault (DMA)	Secure Fault (CPU) MPC Fault (DMA)	MPC Fault	**Allowed**

Table 2-7 CPU or DMA Security Mode to Access Secure or Nonsecure Memory Block

CONFIDENTIAL

A violation (fault) in the MPC can generate an interrupt to a secure interrupt handler and/or generate bus faults to the bus requester. If the bus master is the CPU, the fault will invoke the bus fault handler. For DMA bus masters, the DMA engine will encounter a bus error. MPC will ignore writes and return zeros on a detected violation. MPCs are configured with granularity that target the type of memory, system RAM or RRAM/flash. System RAM uses four MPCs (MPC_SRAMx) assigned to each of the 32 KB SRAM blocks. Each MPC can be partitioned to secure/non-secure areas with a granularity of 256 bytes. A single MPC (MPC_FLS) is assigned to ROM, RRAM and external flash (QSPI) memories and uses a granularity of 2 KB. The MPC_FLS covers the external QSPI flash interface using a set of programmable and fixed configuration elements. This arrangement allows the first 128 KB of flash memory to be configured as secure or nonsecure and the remaining flash is fixed as non-secure.

Peripheral Protection Controllers

The peripheral protection controller (PPC) allows software to partition I/O devices into secure and non-secure address spaces (i.e. worlds). PPCs are similar to MPCs but there is no fine grain control of the memory map. Rather the entire peripheral block can be moved to either the secure or nonsecure worlds. PPCs are bus elements located ahead of each of the ARM peripheral bus expansion ports. They are designed to check bus requests to the port targeting a peripheral on that port. Their location can offer protection from other bus masters such as DMA. A PPC can generate an interrupt or bus fault when a bus address used does not match the configuration for the peripheral. Table 2-8 shows the legal combinations of the CPU or DMA security mode using a secure or nonsecure address alias to access a secure or nonsecure peripheral.

CPU or DMA Channel Mode	Secure Alias to Secure Peripheral	Secure Alias to Non-secure Peripheral	Nonsecure Alias to Secure Peripheral	Nonsecure Alias to Non-secure Peripheral
Secure	**Allowed**	PPC Fault	PPC Fault	**Allowed**
Non Secure	Secure Fault (CPU) PPC Fault (DMA)	Secure Fault (CPU) PPC Fault (DMA)	PPC Fault	**Allowed**

Table 2-8 CPU or DMA Security Mode to Access Secure or Nonsecure Peripheral

CONFIDENTIAL

A violation (fault) in the PPC can generate an interrupt to a secure interrupt handler and/or generate bus faults to the bus requester. If the bus master is the CPU, the fault will invoke the bus fault handler. PPCs will ignore writes and return zeros on a detected violation. The PPC also provides an additional privilege check configurable by software. The secure and nonsecure modes have both privilege and non-privileged states which are orthogonal. Privilege modes are a feature of the ARM CPU and separate from Trustzone. A peripheral can be configured to allow non-privileged access when the bus requester is in either non-secure or secure modes (default is privileged).

Table 2-9, Table 2-10, Table 2-11 and Table 2-12 provide the configuration register and bit position for selecting secure or nonsecure assignment of ATM33/e peripherals. A bit value of ‘1’ sets the peripheral to nosecure space while a ‘0’ (default) puts it in secure space.

- AHB Master Expansion (MSTEXP1) port

Bit	Peripheral
13..10	Reserved
9	BLE Link Controller
8	I2S
7	PDM1 (FIFOs)
6	PDM0 (FIFOs)
5	Reserved
4	Reserved
3	DMA
2	SHA2/HMAC
1	GPIO1
0	GPIO0

Table 2-9 AHBNSPPCEXP1 Register

- APB0 Expansion port

Bit	Peripheral
15	QSPI
14	SLWTIMER
13	SPI1
12	QDEC
11	SPI2
10	KSM
9	Reserved
8	Reserved
7	SPI0
6	PWM
5	UART1
4	UART0
3	WRPR0

CONFIDENTIAL

Bit	Peripheral
0..2	Reserved

Table 2-10 APBNSPPCEXP0 Register

- APB1 Expansion port

Bit	Peripheral
15	Reserved
14	Reserved
13	SHUB
12	RCOS_CAL
11	TRNG
10	GADC
9	SWD
8	PSEQ
7	PMU
6	MDM
5	RADIO
4	RIF
3	WRPR1
2	NVM (OTP Controller)
1	I2C1
0	I2C0

Table 2-11 APBNSPPCEXP1 Register

- APB2 Expansion port

Bit	Peripheral
15..8	Reserved
7	AES
6	CLKRSTGEN
5	I2S

CONFIDENTIAL

Bit	Peripheral
4	Reserved
3	WRPR2
2	Reserved
1	PDM1 (Controller registers)
0	PDM0 (Controller registers)

Table 2-12 APBNSPPCEXP2 Register

2.3 Power Management Unit (PMU)

The power management unit (PMU) provides the core and I/O power supplies to the ATM33/e SoC. The internal power management unit can be bypassed if the required supplies are already available on the system. A single-inductor, multi-input, multi-output switching regulator in the PMU generates the three power supplies used by the SoC: DVDD1P, AVDD1P, VDDIOP, and a fourth auxiliary supply VAUX used internally by the PMU. The PMU block diagram is shown in Figure 2-5. To use all of the PMU supplies, the following connections must be made on the board:

- DVDD1P to DVDD1
- AVDD1P to AVDD1
- VDDIOP to VDDIO

The various voltage rails are brought up in roughly the order shown in Figure 2-3, and takes roughly 7ms to fully regulate all of the rails.

CONFIDENTIAL

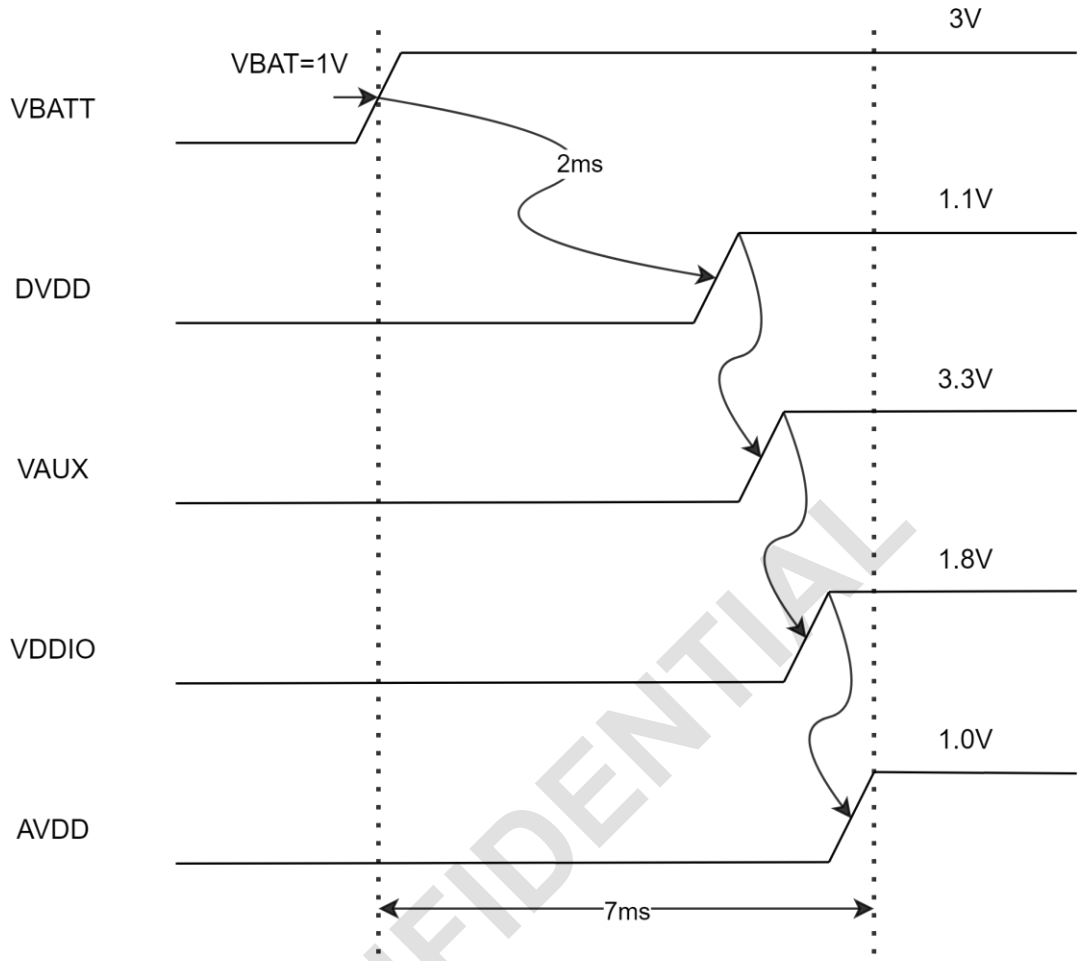


Figure 2-3 Voltage Rail Regulation Sequence

If a PWD is deasserted (in a reset-like mode), a similar sequence is done, and also takes roughly 7 ms, as shown in Figure 2-4.

CONFIDENTIAL

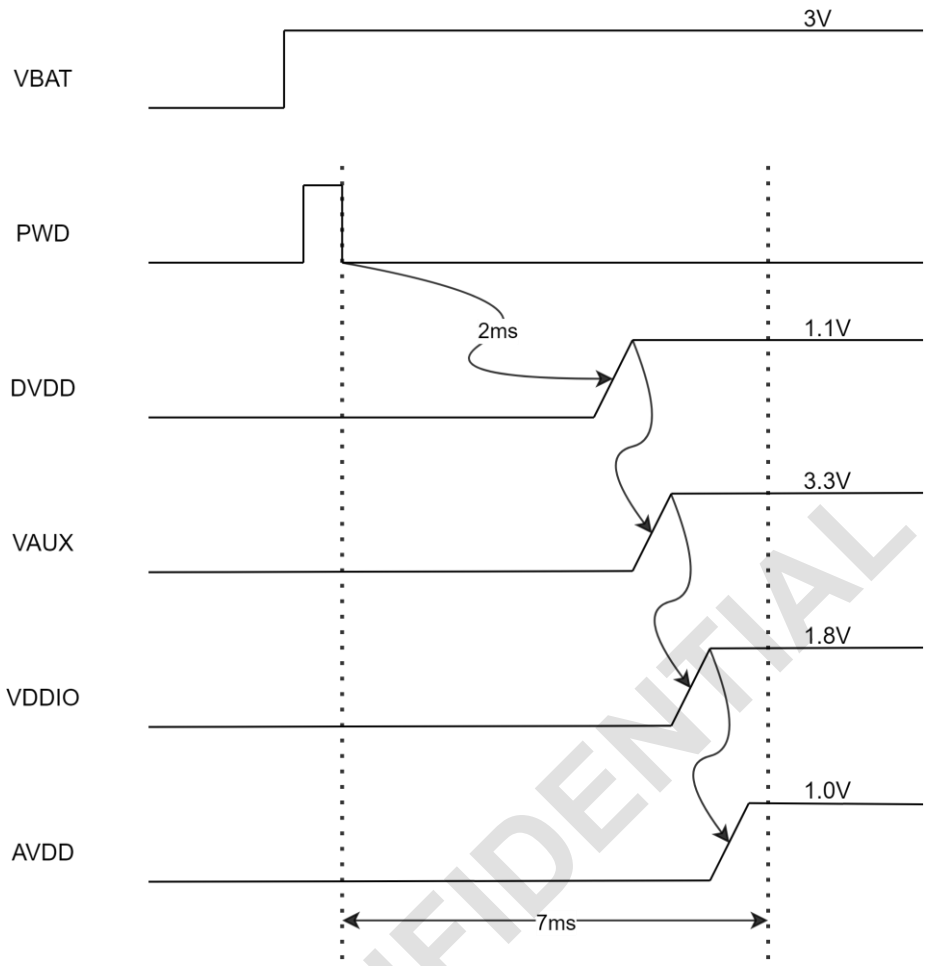


Figure 2-4 Voltage Rail Regulation After PWD Deassertion

The energy harvesting versions of the ATM33e series, denoted by the 'e' (ATM33e), includes an RF harvester whose output can be used by the PMU. The PMU can also be connected to other external sources of harvested energy such as photovoltaic cells, Thermal Electric Generators (TEG), or motion. Harvested energy can extend the battery life or enable operation without a battery for some low duty-cycle applications.

CONFIDENTIAL

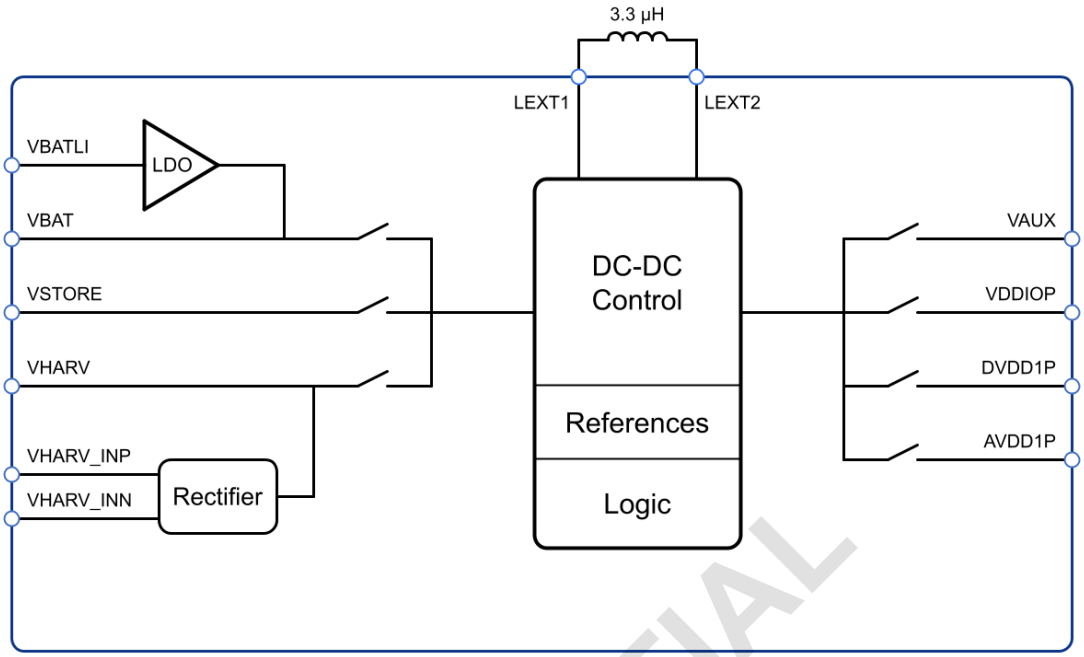


Figure 2-5 PMU Block Diagram

The energy source can be a battery VBAT, a harvester output VHARV, or an energy storage element VSTORE. The switching regulator transfers energy from the harvester output to the storage element when there is excess energy available at the harvester output.

A high-voltage battery or storage device (ie. Li-Ion battery, LiFePO4, or hybrid supercapacitor) can be connected to the VBATLI pin and there is a LDO that regulates VBATLI voltage down to VBAT. The energy is then supplied from VBAT to the rest of the system through the switching regulator.

CONFIDENTIAL

Pin	Description
VBATLI	Battery input for voltage range from 2.7 V to 4.2 V. Must be connected to the battery, external supply, rechargeable storage device or ground (battery-free applications). If VBATLI is not used and VBAT is used, recommend connecting to VBAT. Refer to the datasheet for recommended operating conditions.
VBAT	Battery input for voltage range from 1.1 V to 3.3 V. Must be connected to the battery, external supply, rechargeable storage device or ground (battery-free applications). Must connect to a 10 µF capacitor if VBATLI is used. Refer to the datasheet for recommended operating conditions.
VSTORE	Connection to a large storage capacitor or rechargeable battery. Must be grounded for ATM33 or when the harvester is not used.

Pin	Description
HARV_INN, HARV_INP	Differential RF Harvester Input (negative) Differential RF Harvester Input (positive) HARV_INN and HARV_INP should be grounded when the RF Harvester is not used.
VHARV	Connection to a capacitor of typical value 10 µF. Must be grounded for ATM33 or when the harvester is not used.
LEXT1, LEXT2	Terminals between which the switching regulator inductor is connected.
DVDD1P, AVDD1P, VDDIOP	DVDD1P and AVDD1P are PMU generated digital and analog core supply outputs. VDDIOP is PMU generated 1.8 V I/O supply output.
VAUX	Auxiliary supply output of typical value 3.3 V, used internally by the PMU.
DVDD1, AVDD1	Power supply input for digital and analog core circuits.
VDDIO	Power supply input for digital and analog I/O circuits.

Table 2-13 PMU External Pins

The PMU provides multiple brownout interrupts to enable more reliable operation, especially when using energy harvesting.

2.4 PMU Configurations

The PMU must be configured properly to ensure correct operation. Fields in Table 2-14 must be programmed by the user depending on the desired board configuration:

Register Field	Description
otp_batt_type	0 (default) for battery or a storage device on VBATLI 1 for rechargeable battery or rechargeable storage device on VBAT 2 reserved 3 for non-rechargeable battery on VBAT
otp_vbatt_level	0 (default) for VBAT <= 1.8 V 1 for VBAT > 1.8 V
otp_disable_vddIOgen	0 (default) Set to 1 to disable internal VDDIO generation. VDDIO must be provided externally.
otp_vbatt_brownout	VBAT brownout threshold level If otp_vbatt_level = 0, 0.9 + 0.025*X for X in 0..31 If otp_vbatt_level = 1, 1.4 + 0.05*X for X in 0..31
otp_mppt_type	0 (default) enable digital MPPT

CONFIDENTIAL

Register Field	Description
(ATM3330e)	1 enable analog MPPT for RF harvesting 2 disable all MPPT (voltage regulation)
Otp_vstore_max (ATM3330e)	Max voltage regulated at VSTORE 0 (default) 3.3 V 4 2.5 V 5 2.7 V 6 2.9 V 7 3.1 V

Table 2-14 PMU Configuration Fields

Refer to Table 2-15 for PMU configurations supported by the ATM33/e series.

Battery Options	I/O Supply Option	ATM33	ATM33e
non-rechargeable battery <= 1.8 V on VBAT	External	X	X
	Internal	✓	✓
Non-rechargeable battery > 1.8 V on VBAT	External	✓	✓
	Internal	✓	✓
Li-Ion battery on VBATLI	External	X	X
	Internal	✓	✓
Rechargeable battery > 1.8 V on VBAT*	External	✓	✓
	Internal	✓	✓

Table 2-15 PMU Supported Configuration

* These functionalities will be supported in future software releases.

When using the SDK, CFLAGS makefile argument will program the PMU configuration fields in software. The supported PMU modes of operation break down to the following categories, and will be discussed in detail.

- Supported battery types
- Battery levels

CONFIDENTIAL

- Internal or external IO supply
- System brownout threshold
- Harvesting type
- VSTORE maximum threshold

2.4.1 Supported Battery Types

- Set the supported Battery type by adding the corresponding CFLAGS option in the makefile (or set *otp_batt_type* to the specific value in parenthesis):
    - Li-ion battery on VBATLI (0), `-DBATT_TYPE=BATT_TYPE_LI_ION`
    - Rechargeable battery on VBAT (1): `-DBATT_TYPE=BATT_TYPE_RECHARGEABLE`
      Once this setting is set, VBATLI cannot be accessed.
    - Non-rechargeable battery on VBAT (3): `-DBATT_TYPE=BATT_TYPE_NON_RECHARGEABLE`. Once this setting is set, VBATLI cannot be accessed.
- If this CFLAGS option is not set, the default setting is 3 indicating non-rechargeable battery on VBAT.

2.4.2 Battery Level

- Sets the operating battery level (*otp_vbatt_level)* using the corresponding CFLAGS option in the makefile, or set otp_vbatt_level to the value in parenthesis:
    - VBAT <= 1.8 V (0) : `-DBATT_LEVEL=BATT_LEVEL_LE_1P8V`
    - VBAT > 1.8 V (1) : `-DBATT_LEVEL=BATT_LEVEL_GT_1P8V`
- If this CFLAGS option is not set, the default setting is 1 indicating VBAT>1.8 V.

2.4.3 Internal or External I/O Supply

For applications that cannot support a fixed 1.8 V I/O supply or use external wakeup from SoC Off, option (a) should be used for external I/O supply. For applications that can use a fixed 1.8 V I/O supply for better power consumption or VBAT <= 1.8V, option (b) should be used for internal I/O supply.

(a) One external power supply or battery with external I/O supply:

- Connect VBAT or an external supply to VDDIO
- Connect VAUX to VDDIOP

CONFIDENTIAL

- *otp_disable_vddIOgen* = 1, or add CFLAGS in makefile: `-DVDDIO_TYPE=VDDIO_TYPE_EXTERNAL`

(b) One external power supply or battery with internally generated I/O supply:

- VDDIO = 1.8 V
- *otp_disable_vddIOgen* = 0, or add CFLAGS in makefile: `-DVDDIO_TYPE=VDDIO_TYPE_INTERNAL`

If this CFLAGS option is not set, the default setting is 0 indicating internally generated I/O supply.

2.4.4 System Brownout Threshold

Brownout functionality will be supported in future software releases.

2.4.5 Harvesting Type (**ATM3330e only**)

(a) RF Energy harvesting operation:

- *otp_mppt_type* = 1 for RF harvesting
- Add CFLAGS in makefile: `-DCFG_RF_HARV`

(b) Photovoltaic, Thermal, Mechanical Energy harvesting operation:

- *otp_mppt_type* = 2 for energy sources other than RF.
- Add 2 CFLAGS in the makefile. Once this setting is set, the RF rectifier for harvesting is disabled.

```
-DCFG_NONRF_HARV -DHARV_RANGE=VHARV_RANGE_XPXV_XPXV
```

- VHARV can be regulated from 0.4 V to 2.5 V. Depending on the optimal operation range of the external harvester (ie. PV cell), one of the 16 ranges can be selected.
  - VHARV_RANGE_0P4V_0P45V
  - VHARV_RANGE_0P45V_0P5V
  - VHARV_RANGE_0P525V_0P6V
  - VHARV_RANGE_0P6V_0P7V
  - VHARV_RANGE_0P675V_0P8V
  - VHARV_RANGE_0P75V_0P9V
  - VHARV_RANGE_0P825V_1P0V

CONFIDENTIAL

- VHARV_RANGE_0P9V_1P1V
- VHARV_RANGE_0P975V_1P2V
- VHARV_RANGE_1P05V_1P3V
- VHARV_RANGE_1P175V_1P45V
- VHARV_RANGE_1P3V_1P6V
- VHARV_RANGE_1P5V_1P8V
- VHARV_RANGE_1P7V_2P0V
- VHARV_RANGE_1P95V_2P25V
- VHARV_RANGE_2P2V_2P5V

(c) Combined RF and Photovoltaic harvesting

- *otp_mppt_type* = 2 for combined harvesting.
- Add 3 CFLAGS in makefile: `-DCFG_RF_HARV -DCFG_NONRF_HARV -DHARV_RANGE=VHARV_RANGE_XPXV_XPXV`

CONFIDENTIAL

2.4.6 VSTORE maximum threshold (**ATM3330e only**)

- The storage element connected on VSTORE can be limited to a maximum threshold set by *otp_vstore_max*. Add one of the following options in makefile: `-DVSTORE_MAX=VSTORE_MAX_XPXV`

VSTORE Threshold	*otp_vstore_max* value	Corresponding CFLAGS Setting
2.5 V	4	`VSTORE_MAX_2P5V`
2.7 V	5	`VSTORE_MAX_2P7V`
2.9 V	6	`VSTORE_MAX_2P9V`
3.1 V	7	`VSTORE_MAX_3P1V`
3.3 V	0	`VSTORE_MAX_3P3V`

Table 2-16 Set VSTORE Maximum Threshold

- If not set, the default setting is 3.3 V

Block diagrams for all supported PMU configurations are shown in Table 2-17.

Battery Options	I/O Supply Option	ATM33	ATM33e
non-rechargeable battery <= 1.8 V on VBAT	External	N/A	N/A
	Internal	Figure 2-6	Figure 2-7 for RF,

Battery Options	I/O Supply Option	ATM33	ATM33e
			Figure 2-8 for non-RF
Non-rechargeable battery > 1.8 V on VBAT	External	Figure 2-9	Figure 2-10 for RF, Figure 2-11 for non-RF
	Internal	Figure 2-6	Figure 2-7 for RF, Figure 2-8 for non-RF
Li-Ion battery on VBATLI	External	N/A	N/A
	Internal	Figure 2-12	Figure 2-13 for RF, Figure 2-14 for non-RF
Rechargeable battery > 1.8 V on VBAT*	External	Figure 2-9	Figure 2-10 for RF, Figure 2-11 for non-RF.
	Internal	Figure 2-6	Figure 2-7 for RF, Figure 2-8 for non-RF

Table 2-17 All Supported PMU Configurations

*These functionalities will be supported in future software releases.

CONFIDENTIAL

1) Figure 2-6 supports the following configurations:

   a) Non-rechargeable battery <=1.8 V on VBAT, with internal I/O supply, and no harvesting capability.

   b) Non-rechargeable battery > 1.8 V on VBAT, with internal I/O supply, and no harvesting capability.

   c) Rechargeable battery > 1.8 V on VBAT, with internal I/O supply, and no harvesting capability.

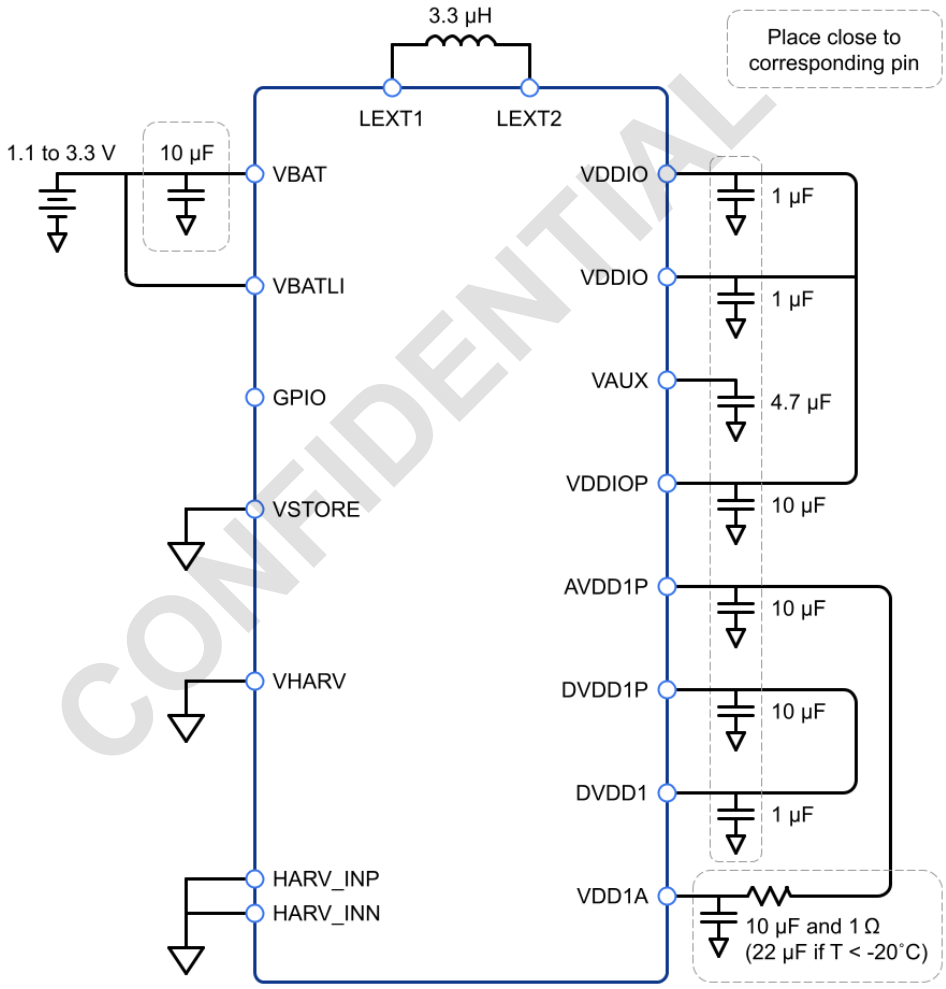


Figure 2-6 - Non-rechargeable or Rechargeable Battery on VBAT, Internal I/O Supply, No Harvesting Capability

CONFIDENTIAL

2) Figure 2-7 supports the following configurations:
   a) Non-rechargeable battery <=1.8 V on VBAT, with internal I/O supply, and RF harvesting source.
   b) Non-rechargeable battery > 1.8 V on VBAT, with internal I/O supply, and RF harvesting source.
   c) Rechargeable battery > 1.8 V on VBAT, with internal I/O supply, and RF harvesting source.

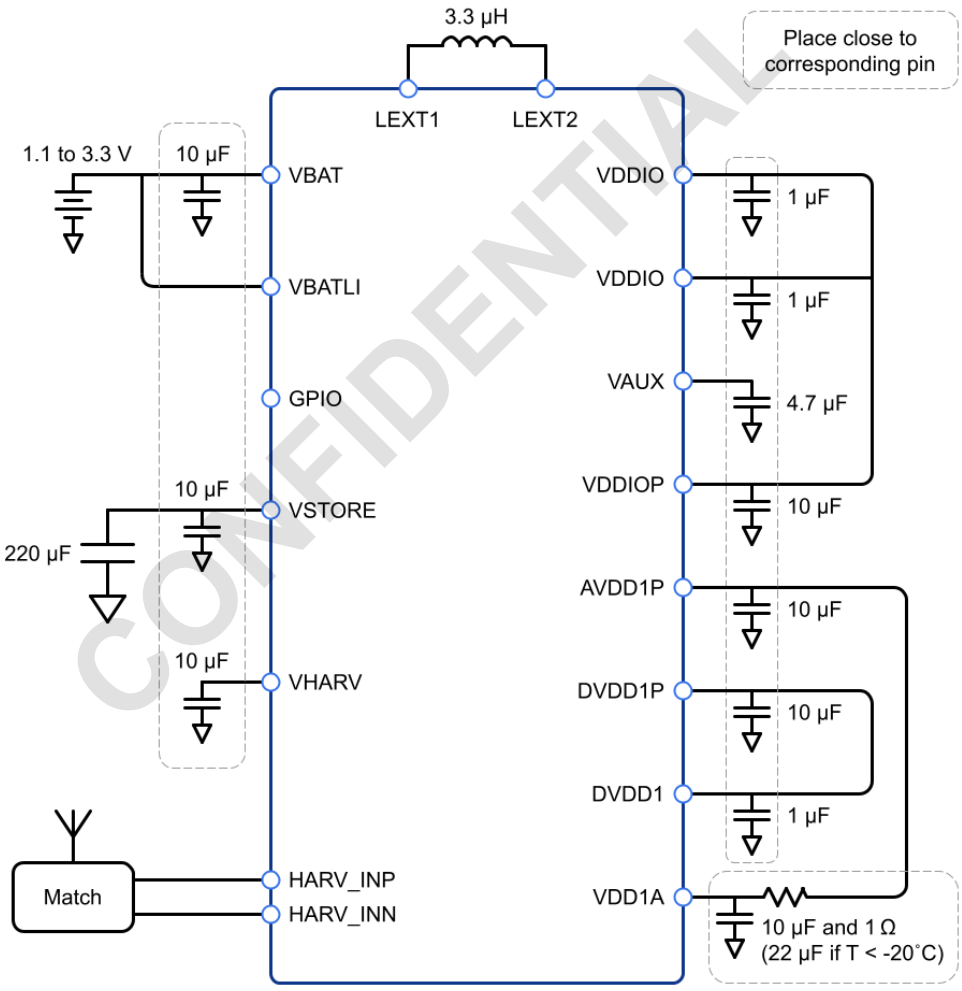


Figure 2-7 Non-rechargeable or Rechargeable Battery on VBAT, Internal I/O Supply, RF Harvesting Source

CONFIDENTIAL

3) Figure 2-8 supports the following configurations:

   a) Non-rechargeable battery <=1.8 V on VBAT, with internal I/O supply, and non-RF harvesting source.

   b) Non-rechargeable battery > 1.8 V on VBAT, with internal I/O supply, and non-RF harvesting source.

   c) Rechargeable battery > 1.8 V on VBAT, with internal I/O supply, and non-RF harvesting source.

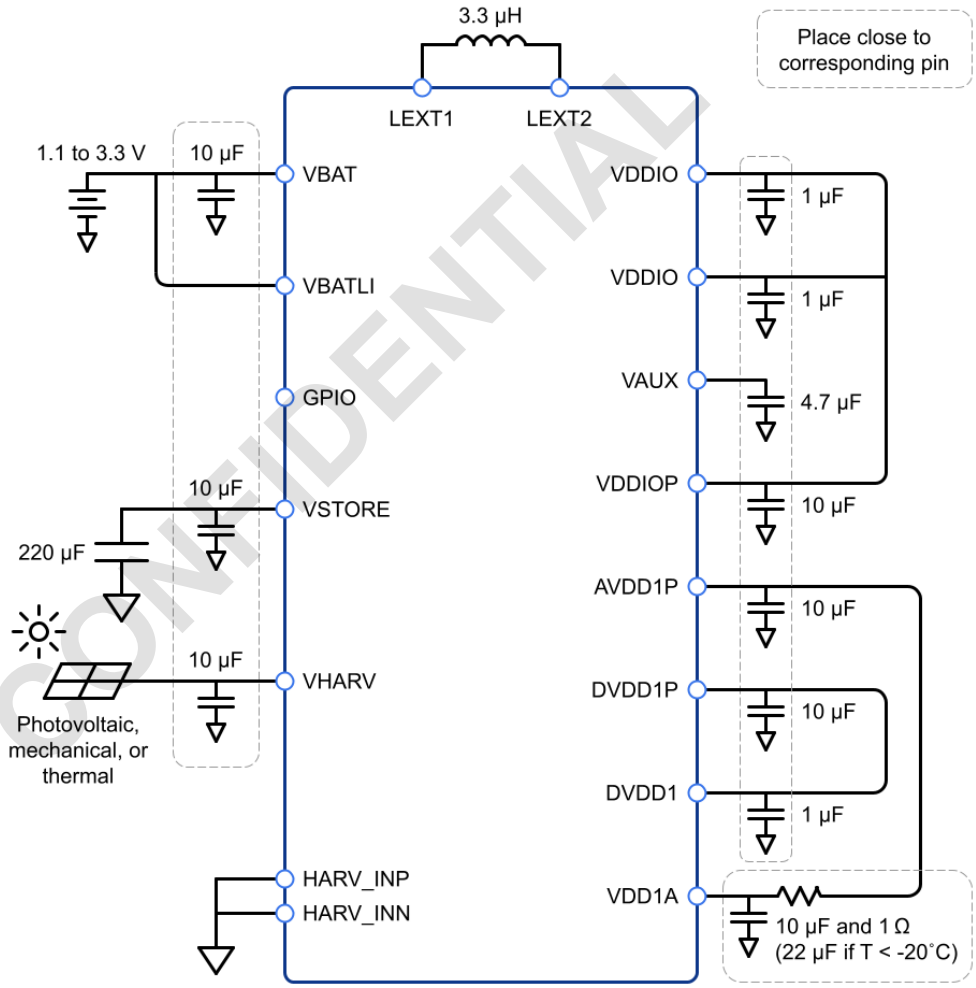


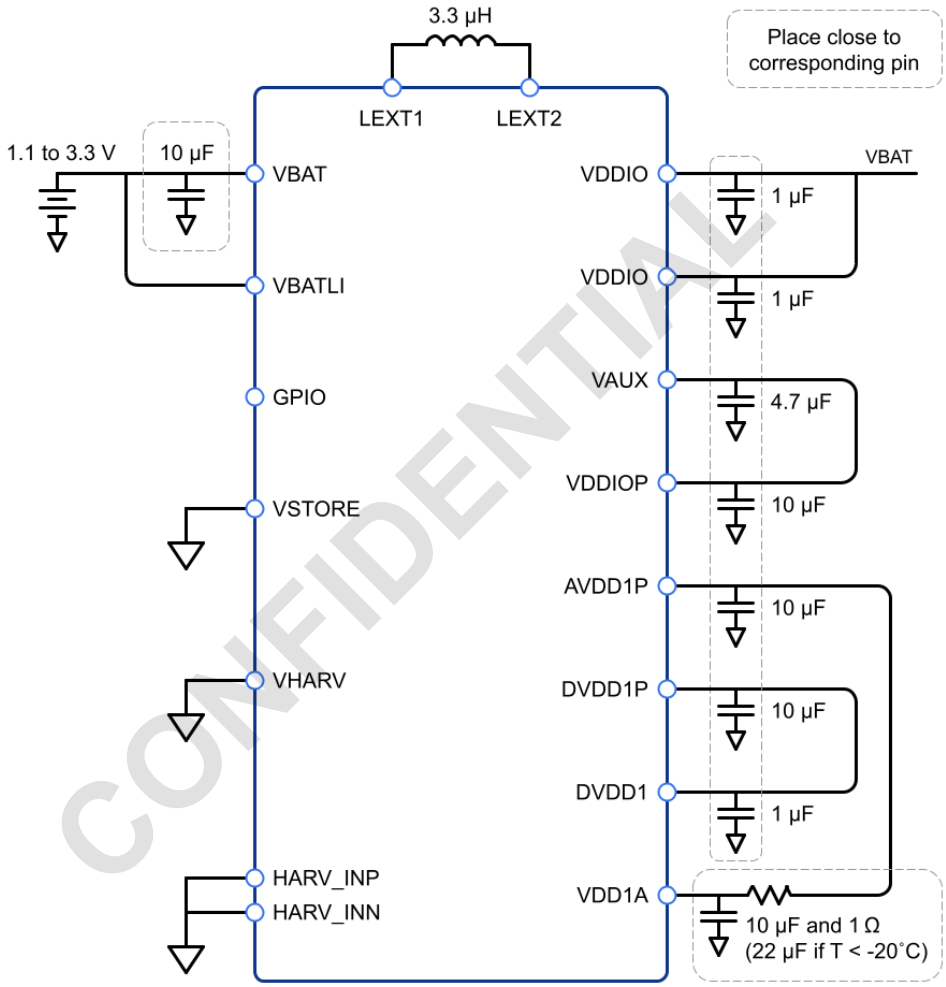
Figure 2-8 Non-rechargeable or Rechargeable Battery on VBAT, Internal I/O Supply, Non-RF Harvesting Source

CONFIDENTIAL

4) Figure 2-9 supports the following configurations:

   a) Non-rechargeable battery > 1.8 V on VBAT, with external I/O supply, and no harvesting capability.

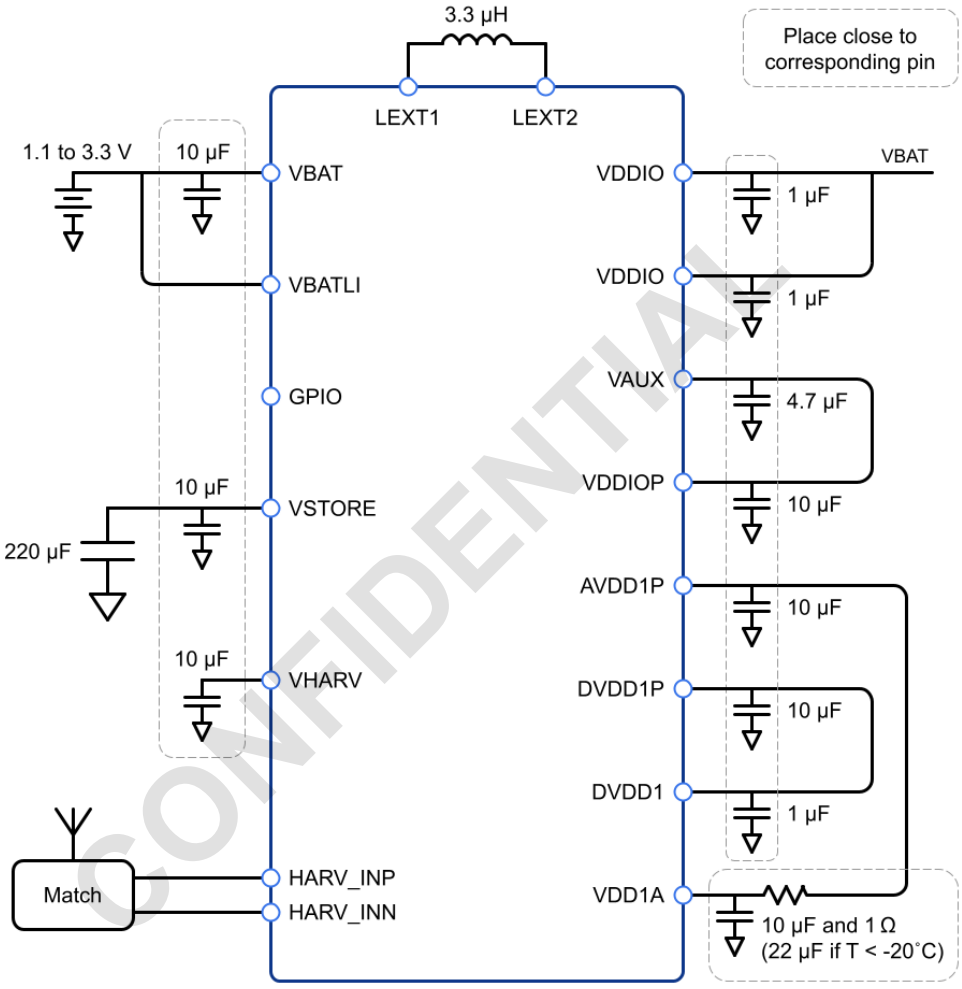
   b) Rechargeable battery > 1.8 V on VBAT, with external I/O supply, and no harvesting capability..



CONFIDENTIAL

Figure 2-9 Non-rechargeable or Rechargeable Battery on VBAT, External I/O Supply, No Harvesting Capability

5) Figure 2-10 supports the following configurations:
   a) Non-rechargeable battery > 1.8 V on VBAT, with external I/O supply, and RF harvesting source.
   b) Rechargeable battery > 1.8 V on VBAT, with external I/O supply, and RF harvesting source.



CONFIDENTIAL

Figure 2-10 Non-rechargeable or Rechargeable Battery on VBAT, External I/O Supply, RF Harvesting Source

6) Figure 2-11 supports the following configurations:
   a) Non-rechargeable battery > 1.8 V on VBAT, with external I/O supply, and non-RF harvesting source.
   b) Rechargeable battery > 1.8 V on VBAT, with external I/O supply, and non-RF harvesting source.

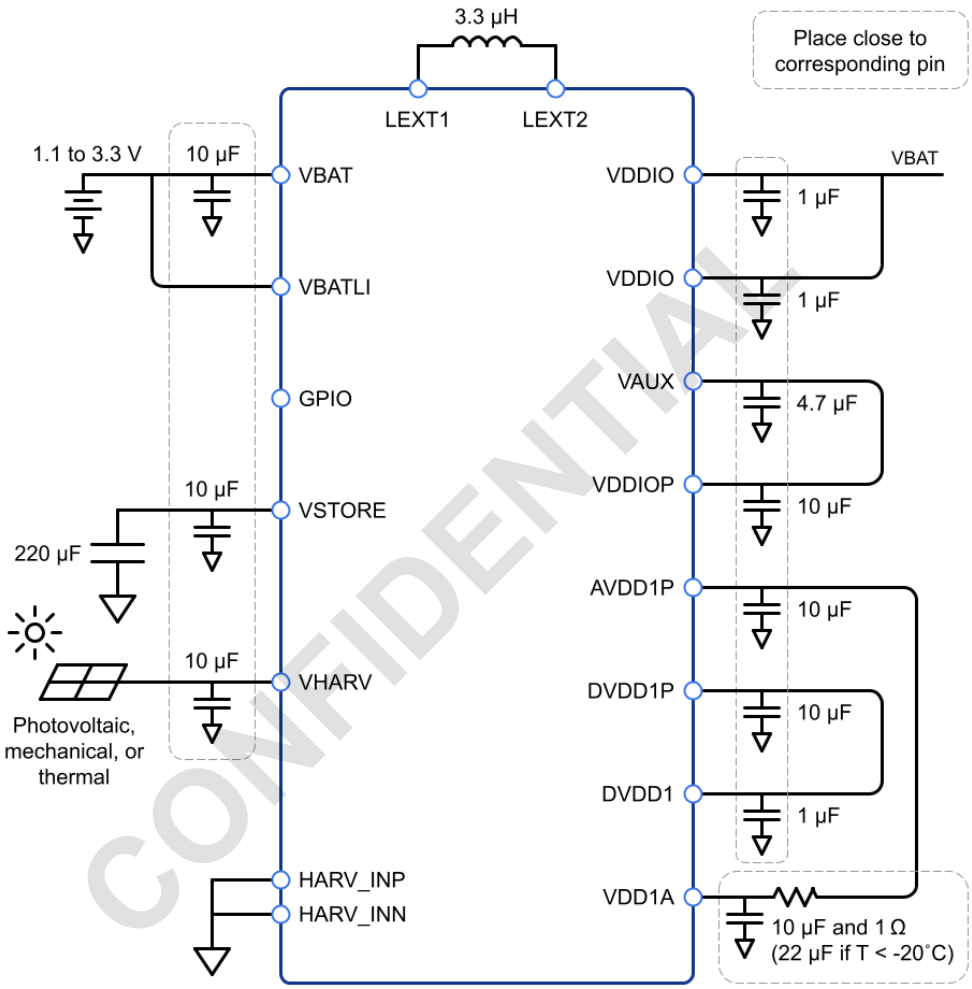
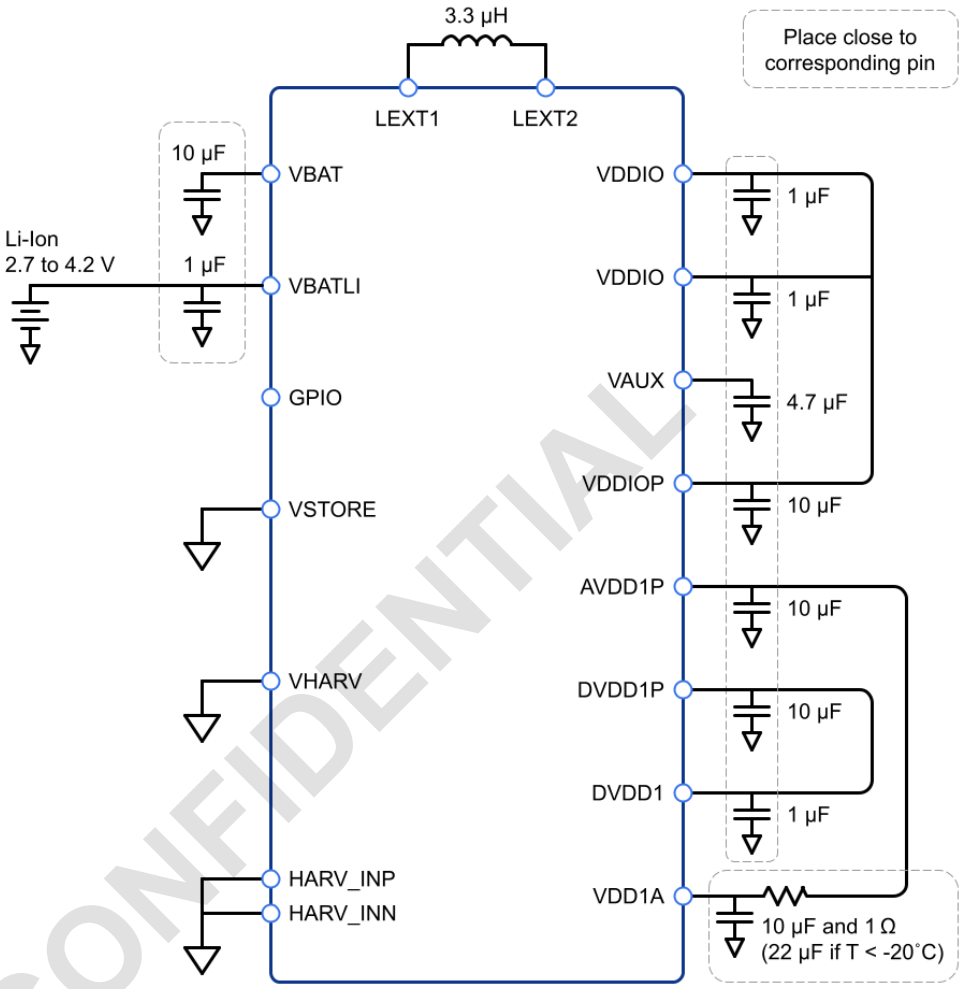


Figure 2-11 Non-rechargeable or Rechargeable Battery on VBAT, External I/O Supply, Non-RF Harvesting Source

CONFIDENTIAL

7) Li-ion battery or storage device on VBATLI with internal I/O supply, and no harvesting source, see Figure 2-12.



CONFIDENTIAL

Figure 2-12 Li-ion Battery or Storage Device on VBATLI, Internal I/O Supply, No Harvesting Capability

8) Li-ion battery or storage device on VBATLI with internal I/O supply, and RF harvesting source, see Figure 2-13.

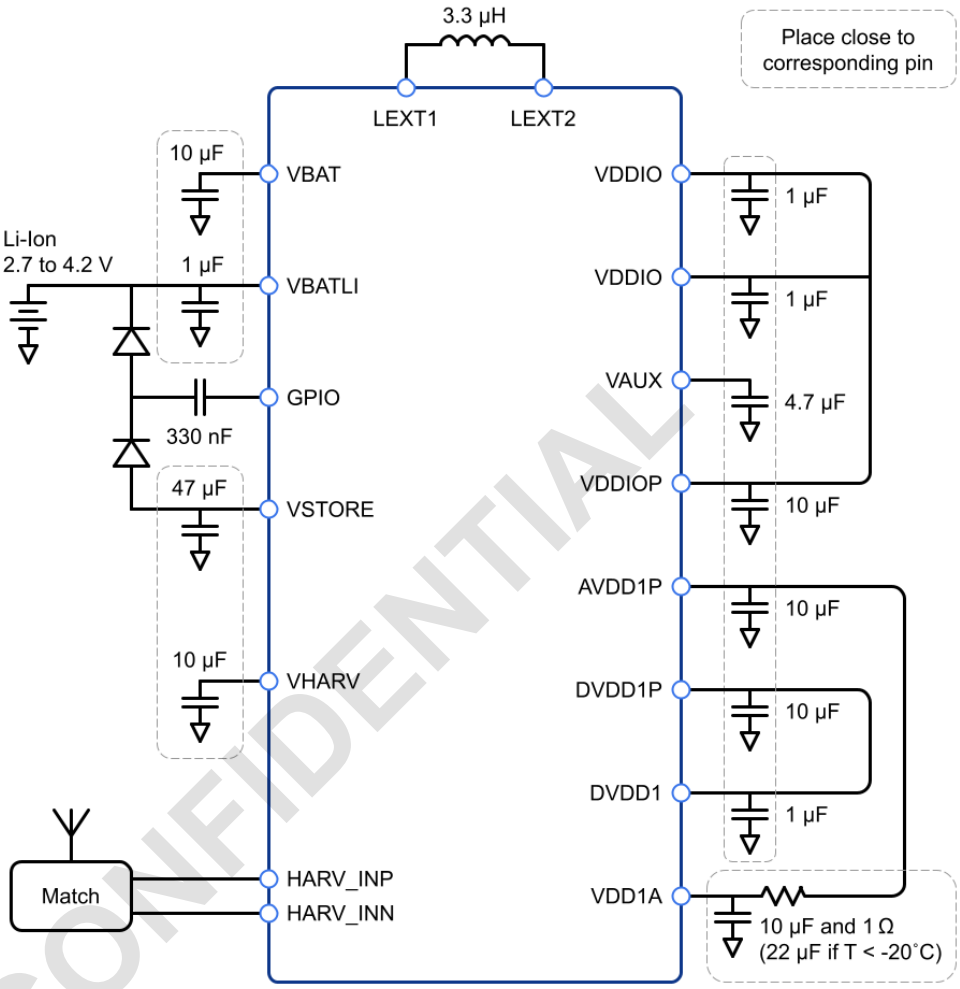
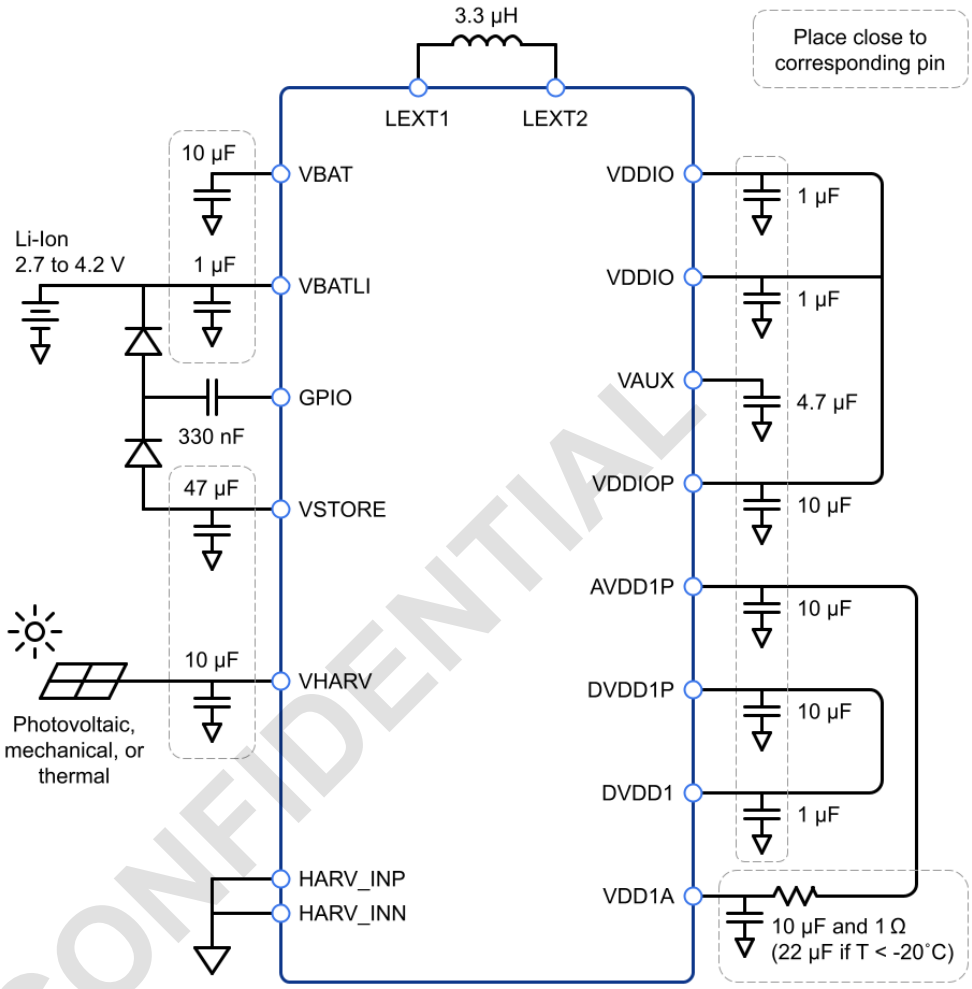


Figure 2-13 Li-Ion Battery or Storage Device on VBATLI, Internal I/O Supply, RF Harvesting Source

CONFIDENTIAL

9) Li-Ion battery or storage device on VBATLI with internal I/O supply, and non-RF harvesting source, see Figure 2-14.



CONFIDENTIAL

Figure 2-14 Li-Ion Battery or Storage Device on VBATLI, Internal I/O Supply, Non-RF Harvesting Source

2.5 Sensor Hub (SHUB)

The sensor hub (SHUB) is a hardware module that, when programmed, can read data from external sensors and write to flash on the quad SPI interface. It can perform this function even when all other power domains are powered down, including the CPU power domain. The sensor hub can trigger a wake up interrupt if any of the read data crosses preprogrammed thresholds.

The sensor hub core, running on a 32.768 kHz CLK_LPC clock, polls data from sensors and transfers them to the flash. Connectivity to sensors is provided by two ports, with

each port configurable as either SPI or I2C protocol. At the system level, the user connects the sensors to the appropriate pins reserved for the sensor hub and programs the pin mux accordingly.

- Port0: The first configuration for port0 is in **port0_setup** register which specifies: the type of protocol (*spi_n_i2c*), the number of sensors connected to this port (*num_sensors*), off/on state of the port (*port_en*) and whether the sensor data are to be compared with pre-programmed threshold quantities (*trig_en*).

  If the protocol is SPI, only one sensor is expected and the user must program the sensor_opcode in the **port0_spi_eng0_ctrl** register and the number of bytes num_bytes in the **port0_sensor0_setup** register to be read from the sensor. Note that an extra byte will be read from the sensor and written to flash, so it should be discarded by the application.

  If the protocol is I2C, up to 8 sensors can be connected to port0 as specified in **port0_setup**.*num_sensors*. For each sensor, the corresponding port0_sensorx_setup register should be programmed with the number of bytes to be read from the sensor minus one. Furthermore, the user must program the control registers for I2C packets, each of which corresponds to one of 4 stages making up for one sequence of I2C data transfer from a sensor. These 4 stages are <sensor id>, <pointer address within the sensor>, <sensor_id> <sensor data> ... <sensor data>, and the last stage (sensor data) can be multiple packets equal to the number of bytes. The ID and address stages are programmed in the corresponding **port0_i2c_idw_x, port0_i2c_addr_x**, and **port0_i2c_idr_x** registers. The data stage is specified in two registers which are common for all sensors: **port0_i2c_dat_ctrl** for the first and intermediate bytes and **port0_i2c_dat_last_ctrl** for the last byte in each read sequence.

- Port1: The first configuration for port 1 is in port1_setup which specifies: the type of protocol (**spi_n_i2c**), the number of sensors connected to this port (**num_sensors**), off/on state of the port (**port_en**) and whether the sensor data are to be compared with pre-programmed threshold quantities (**trig_en**).

  If the protocol is SPI, only one sensor is expected and the user must program the sensor_opcode in the **port1_spi_eng0_ctrl** register, and the number of bytes num_bytes in the **port1_sensor0_setup** register to be read from the sensor. Note that an extra byte will be read from the sensor and written to flash, so it should be discarded by the application.

CONFIDENTIAL

If the protocol is I2C, only one sensor can be connected to port1 and the **port1_sensor0_setup** register should be programmed with the number of bytes to be read from the sensor minus one. Furthermore, the user must program the control registers for I2C packets, each of which corresponds to one of 4 stages making up for one sequence of I2C data transfer from a sensor. These 4 stages are <sensor id>, <pointer address within the sensor>, <sensor_id> <sensor data> ... <sensor data>, whereby the last stage (sensor data) can be multiple packets equal to programmed number of bytes. The ID and address stages are programmed in the **port1_i2c_idw_0, port1_i2c_addr_0**, and **port1_i2c_idr_0** registers. The data stage is specified in two registers: **port1_i2c_dat_ctrl** for the first and intermediate bytes and **port1_i2c_dat_last_ctrl** for the last byte in each read sequence.

2.5.1 Event Schedule

The sensor hub main Finite-State Machine (FSM) is triggered by the PSEQ. Once triggered, the FSM reads the data from the sensors and writes to the flash repeatedly until the PSEQ requests for a stop. The cycle duration is set by sensor hub setup1 register which shall be programmed longer than the time it takes to read all sensors' data and write to flash.

In each cycle (controlled by setup1), sensors on port0 and port1 are read simultaneously. Starting with sensor 0, the FSM reads the data from both ports and writes port 0's data and then port 1's data to the flash. For subsequent sensors, the FSM only receives the data from port 0. Note that, the write phase is broken up to 2 actual flash writes if the bulk of data bytes span page boundaries.

2.5.2 Data Comparison

Port 0 is equipped with one alarm for sensor 0 when only one sensor is connected. Port 1 is equipped with one alarm for the single sensor that can be attached to it. In this section, the alarm on port 0 is used as the reference for description. The alarm treats the stream of bytes from sensor 0 in terms of distinct quantities, each of which is made up of a number of contiguous bytes. The user can define up to 4 quantities which are specified in the **port0_alm0_quan_ctrl** register by their size and starting positions in the data stream (up to 32 bytes). The quantities cannot overlap, and each quantity can be a

CONFIDENTIAL

maximum of 3 bytes, so the combined number of bytes is a maximum of 12 bytes.

- If the quantities have their most significant bytes first, the user must program (**port0_alm0_quan_ctrl**.*lsb_first*) to 0.
- **port0_alm0_quan_ctrl**.*quan_dat_is_signed* specifies whether the quantities are signed numbers.
- Next, set the floor and ceiling reference quantities with which the sensor's quantities are compared. 12 bytes are allocated for the floor and the ceiling. For the ceiling reference, the 12 bytes, *byte_max_00.... byte_max_11*, are spread across registers **port0_alm0_thrhld_max_0, port0_alm0_thrhld_max_1** and **port0_alm0_thrhld_max_2**. For the floor reference, the 12 bytes, *byte_min_00.... byte_min_11*, are spread across registers **port0_alm0_thrhld_min_0, port0_alm0_thrhld_min_1 and port0_alm0_thrhld_min_2**.
- For each reference, the hardware packs the bytes together into quantities in similar fashion as it does to the sensor's quantities. Each reference quantity is determined by the sensor quantity size. For instance, if quan0_size = 1, quan1_size = 2 , quan2_size = 2 and quan3_size = 0, then the groupings are as follows:
    - {byte_min_00, byte_min_01}, {byte_min_02, byte_min_03, byte_min_04}, {byte_min_05, byte_min_06, byte_min_07}, {byte_min_08}
    - {byte_max_00, byte_max_01}, {byte_max_02, byte_max_03, byte_max_04}, {byte_max_05, byte_max_06, byte_max_07}, {byte_max_08}
- All 4 sensor quantities are then compared to their respective references and have their out of range status captured in **port0_status.out_of_range_stat**. The total number of possible values of the 4-bit *out_of_range_stat* is 15 (disregarding 0). The user can choose any value to trigger the CPU by programming the desired value in **port0_alm0_trig_ctrl**.
- Last, each port has its trigger enable in **portx_setup**.*trig_en*

CONFIDENTIAL

2.5.3 External Flash Control

The sensor data is written to the external flash. The user must specify the following items for the flash operation:

- *Opcodes* in **flash_ctrl0** register for write enable, page program, power down and release from power down. Set **flash_ctrl3**.*is_macronix* and **flash_ctrl3**.*serialize_pp_address* depending on the flash vendor.
- Start address in **flash_ctrl1** register. This flash address is where the first data byte is written to.
- Max addr in **flash_ctrl2** register. This is the last address for sensor data. The sensor hub FSM will wake up the CPU when this address is reached. The number of leftover data bytes are stored in lost_write_cnt in flash_status1 register and **flash_status0**.*wr_saturated* is set by the hardware. These status registers can be cleared by writing ones to **flash_ctrl3**.*clear_lost_write_cnt* and **flash_ctrl3**.*clear_flash_overflow*, respectively.
- Interface mode in **flash_ctrl3**.*qspi_mode* determines whether the interface is single SPI or quad SPI.
- Page program delay time in **flash_ctrl4**.*pp_delay*. After a page program opcode, the flash takes some time to complete the write process.
- Power cycling mode: This mode is controlled by setting **flash_ctrl3**.*power_cycle_en* to one. In this mode, the flash chip is put into a low power state after each write and recalled to active state at the start of the sensor read phase. To account for the flash recovery time from the low power state, the user can program for a delay time in **flash_ctrl4**.*pd_delay*. The write phase only starts if both the read phase and delay flash_ctrl4.pd_delay are completed. For Winbond and Micron, set **flash_ctrl3**.*rpd_has_clock* = 1.

CONFIDENTIAL

2.5.4 Storage in SRAM

To store the data in SRAM, the following steps are taken before the start of SHUB activity:

1) set the configuration bit store_in_sram in SHUB

2) set the appropriate PSEQ configurations bits *shub_sram0_en*, *shub_sram1_en*, *shub_sram2_en*, and *shub_sram3_en*. Each bit allows SHUB the write access to a block of sram.

3) set wr_start_address to point to the starting address of a location in sram.

2.5.5 Advertisement with SHUB

In ATM33/e, the sensor hub microcontroller has the capability to direct the BLE link controller to transmit non-scannable/nonconnable advertisements.

In this mode, each cycle is made up of two phases: data collection and BLE advertisement.

The sensor hub is equipped with a small engine to perform memory read/write operations to the BLE link controller.

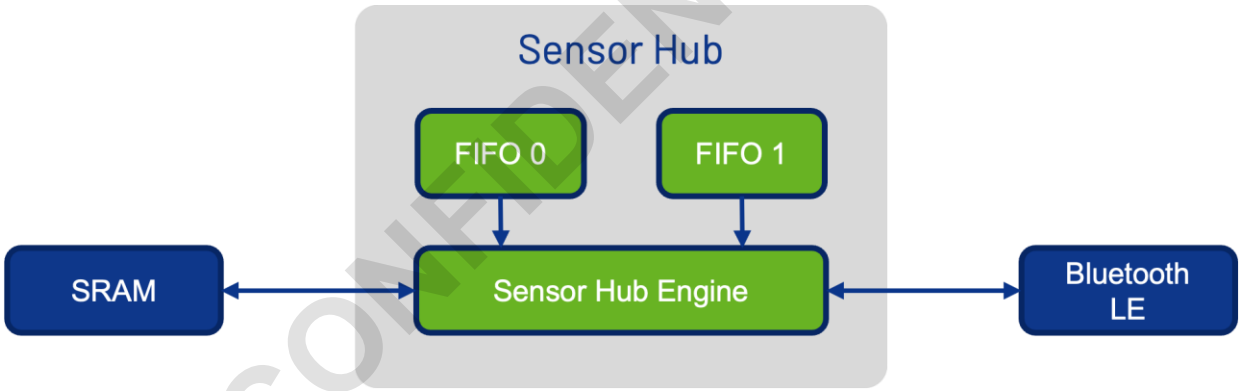


Figure 2-15 Sensor Hub Block Diagram

CONFIDENTIAL

The instruction code for the sensor hub engine is stored in the SRAM. By executing the code, the sensor hub engine performs the following tasks:

1) accesses the fifo buffers that store the sensor data.

2) configure the Bluetooth LE link controller to transmit advertisement packets. The packets may contain sensor data.

3) trigger the PSEQ FSM to start the next cycle of sensing/ble_adv or end the low power state and wake up the MCU.

It takes around 400 words to perform the tasks.

Limitations of the microcontroller in this mode:

4) No data storage in this mode.

5) On interface 0, instead of up to 8, only one external sensor device is supported.

2.6 Bluetooth LE Radio

The block diagram of the low-power radio is shown in Figure 2-16. The radio supports Bluetooth 5.3 including 2 Mbps high-speed PHY, 1 Mbps basic PHY, 500 kbps and 125 kbps PHY. Refer to the Bluetooth LE Coded PHY white paper for more information.

The basic 1 Mbps PHY is compatible with Bluetooth 4.x, while the 2 Mbps rate provides 2X speed and the long range rates provide up to 4X range. The transmit path uses digital direct frequency modulation of a fractional-N synthesizer to create a constant amplitude GFSK signal that is amplified by a power amplifier to provide the desired RF output level. On the receive path, an incoming RF signal is first amplified by an LNA before downconversion to baseband and digitized by two successive-approximation analog-to-digital converters. The digitized signal is sent to the Modem for further digital processing. The radio architecture is optimized for burst data transmission using Frequency-Hopping Spread Spectrum (FHSS) with 40 channels with 2 MHz spacing (3 advertising channels/37 data channels). Only a single RF Input/Output pin is needed, thereby simplifying board-level design.

CONFIDENTIAL

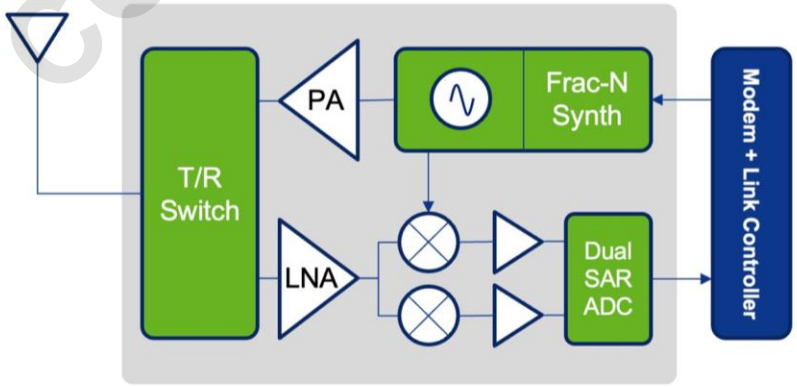


Figure 2-16 Bluetooth LE Radio Block Diagram

2.6.1 Link Controller

The Bluetooth 5.3 link controller (LC) and host stack provides interface between the microcontroller (MCU), modem (MDM) and exchange memory (EM), allowing the MCU

to access through AHB bus to the control registers and exchange memory. During transmit, software writes the packet payload and control structures into the exchange memory. The link controller serializes the data into a bit stream to the modem. During receive, the operation is reversed. Received data from the modem is processed and stored in the exchange memory to be read by software.

2.6.2 LC Module Description

- Interrupt Control handles interrupt to the MCU.
- Wi-Fi Coexistence follows the 3-wire packet traffic arbitration protocol to ensure minimal interference between Wi-Fi and Bluetooth LE radio.
- AHB interface translates AMBA AHB bus slave signals to control and data access to registers and exchange memory.
- Control Reg contains control and status registers.
- Timer interfaces to the lower power control module which resides in always on power domain and can wake up Bluetooth LE LC on timeout or wakeup event detection.
- Crypto Proc encrypts and decrypts payload based on AES-CCM.
- Bit Proc performs bit stream operations like whitening/de-whitening, CRC generation/check, preamble generation.
- Core
    - Mem Control arbitrates memory access requests from various masters
    - Event Schloss starts an event by pref-etching entries in exchange table in exchange memory.
    - Event Control reads and writes control structures and descriptors from and to exchange memory.
    - Pkt Control controls the sequence of operations performed by the bit processor.
    - Srch Eng parses white list or periodic advertiser list and ADI list stored in exchange memory.
    - RAL Srch Eng is in charge of private address resolution and generation, managing RAL structure stored in exchange memory.
    - Freq Control calculates the frequency to be used for transmission and reception.

CONFIDENTIAL

2.6.3 Modem

The modem (MDM), along with the radio, link controller and software stack, forms a highly efficient Bluetooth 5.3 solution. Transmission and reception at rates of 2 Mbps, 1 Mbps, 500 kbps and 125 kbps are supported.

Receiver

The modem and radio are enabled prior to expected packet reception as determined by the link controller. Baseline power measurements of the incoming signal strength are made which serve as a reference for future measurements. Packet detection is accomplished through a combination of power measurements, signal saturation, and correlation with the expected Bluetooth LE preamble. Analog receive gain is adjusted based on the incoming signal strength. Initial estimation of carrier frequency offset and timing synchronization is performed early in the packet, followed by continuous tracking throughout. Correlation with the expected access address is performed and is indicated to the link controller if found. A Received Signal Strength Indicator (RSSI) is also sent to the link controller if the access address is found.

The modem and radio receiver are turned off by the link controller at the end of a completed packet or through various error conditions. The modem may initiate a restart of the packet detection process if the access address is not found, or if synchronization is not achieved.

Channel information is provided directly from the link controller to the radio without involvement of the modem.

Transmitter

There is minimal involvement of the modem in the transmit process. The link controller provides the symbol bit stream, which is then shaped consistent with the GFSK requirements to provide a frequency deviation for the radio.

Channel information and target transmit power are provided directly from the link controller to the radio.

CONFIDENTIAL-

2.7 Wakeup Receiver

Wakeup receiver (WURx) is a feature in ATM33 that could further lower the power consumption. When enabled, the WURx allows the system to stay in sleep mode while listening to incoming RF signals without using the BLE radio. Only when the correct Rx ID is detected does the main radio turn on. This allows for a much lower power consumption since the WURx circuit uses very little power and much less than the BLE radio. In the case of ATM33, it uses only about 400 nA. And because the WURx is always listening, it allows for low latency. Latency of the wakeup receiver is typically in the order of 20 ms to 1 s, depending on the length of the Rx ID code used to identify the target device. Sensitivity is -55 to -60 dBm depending on the power consumption setting. WURx is intended for short range.



Figure 2-17 Wakeup Receiver Application Block Diagram

Asset beacon is a tag device based on Atmosic ATM33 SoC that includes a RF wakeup receiver. Depending on the use case and availability of the wakeup source (access point) different wakeup scenarios can be supported.

Access point can be any of the following:

1) A hub device based on a Bluetooth LE SoC, such as Atmosic ATM2/ATM3/ATM33 or other BLE standards-compliant chips. Access to the lower levels of the software stack is required to configure high duty cycle advertising beacons.
2) Phone, tablet, or other mobile devices that can send Bluetooth inquiry signals.
3) Any software-defined/programmable radio device that can send RF signals with predefined on-off patterns. The transmitted RF energy is treated as OOK (On-Off Keying) pattern.

CONFIDENTIAL

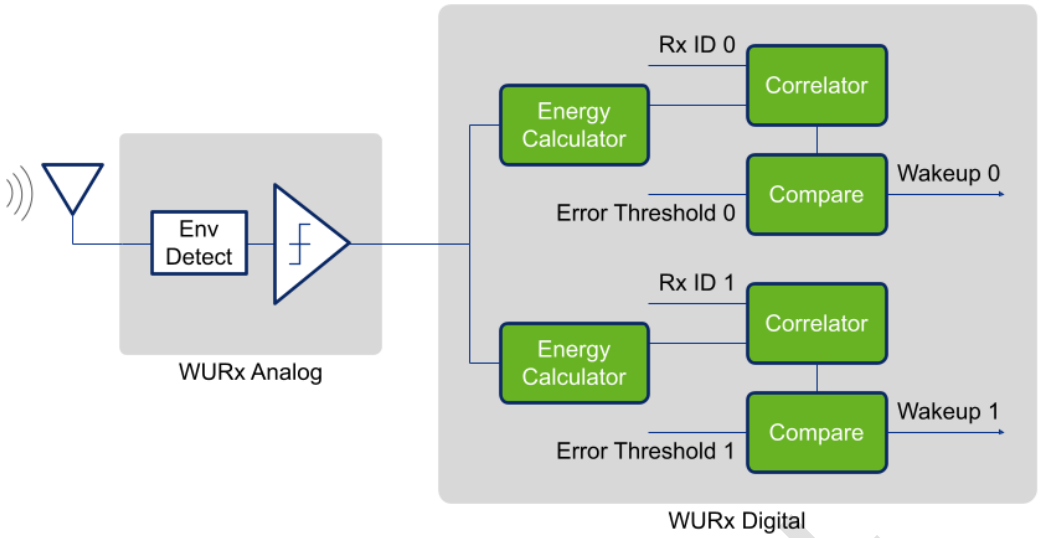


Figure 2-18 Wakeup Receiver Block Diagram

WURx is an OOK receiver and uses an envelope detector to find specific patterns in the incoming signal. The best way to generate the ID is to use an OOK transmitter. The pattern of the OOK signal, will correspond to a stream of ones and zeros that the digital part of WURx would check against the Rx ID it is expecting. If transmitting a custom OOK signal isn't possible, any RF signal with the right RF frequency can be used as an OOK signal as long as its length and duty cycle is suitable to match a reasonable ID when the envelope is sampled by a 32 kHz clock. Some examples of available RF sources are iBeacon or BT inquiry.

WURx is optimized to operate in the 2.4 GHz Bluetooth frequency band so that it can respond to a wakeup signal based on Bluetooth advertising packets. However, the wakeup receiver can work anywhere from 400 MHz to 2.5 GHz if the RF front end match is appropriately modified.

Wakeup scenarios

When a dedicated transmitter is available, ATM33's WURx is capable of decoding an Rx ID that includes three groups of identification: global ID, group ID, and individual ID. This is desirable when there are many ATM33's in the listening range of the access point and the user can wake up all, some, or only one of them. If using iBeacon, BT inquiry, or any other predefined protocol, we need to have a two stage wake up where the main BLE transceiver should decide is the wake up was intended or not

WURx also supports staged wakeups which is useful when not using a dedicated transmitter. In this mode, after the WURx detects a Rx Id match, the BLE radio will determine if the wakeup is correct. The wakeup could be a global or target wakeup. In global wakeup, the radio is turned on and it transmits beacons. In target wakeup, the

CONFIDENTIAL

radio only scans, and if the BD address is found, it'll then transmit beacons. In both cases, the chip will go back to sleep if it was a false wakeup; however, the WURx will keep listening for another wakeup signal.

As shown in figure 2-18, ATM33 can listen for two completely different Rx IDs simultaneously. This is especially useful when listening to access points that could be either iOS or Android.

Filtering false wakeups

One of the factors that increases the average power consumption for wakeup receivers is false wake up. RF wakeup receiver is an OOK frequency detector with relatively poor interference rejection. If the environment is too busy, the wakeup receiver can wake up on other signals, causing false alarm wakeup events. To prevent too many false wakeups, the user can enable a filter that helps prevent most false wakeups. To use the filter, the access point should be programmed to send the wakeup signal (BL inquiry for example). The WURx's filtering process will expect a certain number of Rx IDs with a known delay between each one before a wakeup event is declared. The filtering process will significantly lower the number of false wakeups.

CONFIDENTIAL

2.8 Pin Multiplexing

In the ATM33/e Series of SoCs, each package will have a number of programmable I/O pins that can support multi-function signals, as shown in Table 2-18.

Chip and Package	Total I/Os	I/O Description
ATM3330/ATM3330e 7x7 mm QFN	31	P0-P30
ATM3330e 5x5 mm QFN	15	P0, P1, P4, P5, P15, P17-22, P25, P27, P29, P30
ATM3325 5x5 mm QFN	21	P0, P1, P3-7, P12-15, P17-22, P25, P27, P29, P30
ATM3325 5x5 mm QFN with Extended Storage	18	P0, P1, P3-5, P7, P8, P14, P17-22, P25, P27, P29, P30.
ATM3325 WLCSP	21	P0, P1, P3-7, P12-15, P17-22, P25, P27, P29, P30

Table 2-18 Total Number of I/Os

Multiplexing of the I/O pins are shown in Table 2-19.

Function	P0	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13	P14	P15	P16	P17	P18	P19	P20	P21	P22	P23	P24	P25	P26	P27	P28	P29	P30
swdclk	x																														
swdio		x																													
qspi_clk	x						x						x						x												x
qspi_csn		x						x						x						x						x					
qspi_d0			x						x						x						x						x				
qspi_d1				x												x						x						x			
qspi_d2					x												x						x						x		
qspi_d3						x												x						x						x	
spi0_csn	x								x								x				x										x
spi0_sclk		x																x								x		x			
spi0_mosi			x																x								x		x		
spi0_miso				x																x										x	
spi1_csn					x								x								x										
spi1_sclk						x								x								x									
spi1_mosi							x								x								x								
spi1_miso								x								x								x							
i2c0_sck	x				x			x					x				x				x						x				x
i2c0_sda		x				x								x				x				x				x		x		x	
i2c1_sck			x				x								x				x				x				x		x		
i2c1_sda				x				x								x				x				x						x	
pdm0_clk	x				x				x				x				x				x										x
pdm0_in		x				x								x				x				x				x		x			
pdm1_clk			x				x								x				x				x				x		x		
pdm1_in				x				x								x				x				x						x	
i2s0_sck_out	x			x			x						x			x			x			x						x			x
i2s0_ws_out		x			x			x						x			x			x			x			x			x		

CONFIDENTIAL

i2s0_sd_out			x			x			x						x			x			x			x				x		
i2s0_sck_in	x			x			x						x			x			x			x						x		x
i2s0_ws_in		x			x			x						x			x			x			x				x				
i2s0_sd_in			x			x			x						x			x			x			x				x			
uart0_rx	x		x		x		x		x				x		x		x		x		x			x				x		x		x
uart0_tx		x		x		x		x						x		x		x		x		x		x				x		x		
uart0_cts	x				x				x				x				x				x										x	x
uart0_rts		x				x								x				x				x					x			x		
uart1_rx		x				x								x				x				x					x			x		
uart1_tx			x				x								x				x				x					x	x	x		
uart1_cts				x				x								x				x				x							x	
uart1_rts	x				x				x				x				x				x											x
pwm0	x					x			x				x			x																x
pwm1		x															x										x		x			
pwm2			x															x												x		
pwm3				x															x												x	
pwm4					x								x							x												
pwm5						x								x								x										
pwm6							x								x								x									
pwm7								x								x								x								
ksi19																												x				
ksi18																													x			
ksi17																			x													
ksi16																		x														
ksi15																	x															
ksi14																x																
ksi13															x																	
ksi12													x														x					
ksi11												x																		x	x	

CONFIDENTIAL

ksi10											x												x					x		
ksi9										x													x					x		
ksi8									x													x								
ksi7								x													x									
ksi6							x													x										
ksi5						x													x											
ksi4					x													x												
ksi3				x													x													
ksi2			x													x														
ksi1		x													x															
ksi0	x													x													x			
kso19																											x			
kso18																												x			
kso17																															x
kso16																														x	
kso15																													x		
kso14																								x							
kso13																							x								
kso12												x													x						
kso11											x																			x	x
kso10										x													x					x		
kso9										x													x					x		
kso8									x													x								
kso7								x													x									
kso6							x													x										
kso5						x													x											
kso4					x													x												
kso3				x													x													
kso2			x													x														

CONFIDENTIAL

kso1		x													x														
kso0	x													x													x			
shub_spi0_csn	x								x								x				x									x
shub_spi0_sclk		x																x								x		x		
shub_spi0_mosi			x																x								x		x		
shub_spi0_miso				x																x										x	
shub_spi1_csn					x								x								x										
shub_spi1_sclk						x								x								x									
shub_spi1_mosi							x								x								x								
shub_spi1_miso								x								x								x							
shub_i2c0_sck	x				x				x				x				x				x										x
shub_i2c0_sda		x				x								x				x				x				x		x		x	
shub_i2c1_sck			x																x												
shub_i2c1_sda				x																x											
qdec0_a	x						x						x						x												x
qdec0_b		x						x						x						x							x				
qdec1_a			x						x						x						x							x				
qdec1_b				x												x						x							x			
qdec2_a					x												x						x							x		
qdec2_b						x												x						x							x	
xlna_xpa0	x					x										x					x						x			x		
xlna_xpa1		x					x										x					x						x			x	
xlna_xpa2			x					x					x					x						x								
xlna_xpa3				x					x					x					x						x							
xlna_xpa4					x										x					x									x			x
mdm_in4																													x			
mdm_in3									x																							
mdm_in2								x																								
mdm_in1							x																									

CONFIDENTIAL

mdm_in0						x																							
mdm_out7																									x				
mdm_out6																								x					
mdm_out5																													x
mdm_out4																						x							
mdm_out3																					x								
mdm_out2																				x									
mdm_out1																			x										
mdm_out0																		x											
lc_rx_en	x		x		x		x		x				x		x		x		x		x		x				x		x		x
lc_tx_en		x		x		x		x						x		x		x		x		x		x		x		x		x	
ant_out7								x							x							x								
ant_out6							x							x							x									
ant_out5						x							x							x										
ant_out4					x							x							x											
ant_out3				x														x										x		
ant_out2			x														x								x		x			
ant_out1		x														x								x		x				
ant_out0	x								x						x														x	
ble_sync	x																		x								x		
ble_in_process		x																		x								x		
ble_tx			x									x									x									
ble_rx				x									x									x								
ble_pti0					x									x															x	
ble_pti1						x									x									x						
ble_pti2							x									x									x					
ble_pti3								x									x													
wlan_tx									x									x												
wlan_rx																			x							x				

CONFIDENTIAL

tracedata_iq3					x								x								x									
tracedata_iq4						x								x								x								
tracedata_iq5							x								x								x							
tracedata_iq6								x								x								x						
tracedata_iq7	x								x								x													x
tracedata_iq8		x																x								x		x			
tracedata_iq9			x																x								x		x		
tracedata_iq10				x																x										x	
tracedata_iq11					x								x								x										
tracedata_iq12						x								x								x									
tracedata_iq13							x								x								x								
tracedata_iq14								x								x								x							
tracedata_iq15	x								x								x														x
dbg	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
dtopbyp	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
gpio	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x

Table 2-19 Multiplexed Pin Table

CONFIDENTIAL

Refer to the Pin Assignment and Pinmux Programming White Paper available on the Atmosic Support website for further guidance. Using the Atmosic Pinmux Tool in the SDK is recommended for selecting digital I/O functionality.

The user must program the selection control per pin correctly to connect the pin to the intended functional signal. A set of registers encapsulates the selection control bits for all the pins. Each pin has a 8-bit register field which is used to select which function is applied to the physical pin. Table 2-20 summarizes the mapping between the selection control bits to each pin.

Register Name	Bit 31:24	Bit 23:16	Bit 15:8	Bit 7:0
PSEL_A	P3	P2	P1	P0
PSEL_B	P7	P6	P5	P4
PSEL_C	P11	P10	P9	P8
PSEL_D	P15	P14	P13	P12
PSEL_E	P19	P18	P17	P16
PSEL_F	P23	P22	P21	P20
PSEL_G	P27	P26	P25	P24
PSEL_H	Reserved	P30	P29	P28

Table 2-20 Pin Selection Control to Pin Mapping

CONFIDENTIAL

Each I/O has a pull-up or pull-down setting, which enables a 125 kΩ pull-up or pull-down resistor. This setting can be found by enabling the corresponding bits in **wrpr0**.*pupd_ovrd* and **wrpr0**.*pupd_ovrd_val*. Please see the register specification document for more details.

2.9 Program Sequencer (PSEQ)

PSEQ provides mechanisms for the user application to enter and leave the low power states: Retention, Hibernation, and SoC Off.

Wake from Hibernation or Retention is achieved from a variety of possible sources, and any combination of the possible sources is possible. Use **hib_wake_mask** or **retain_all_wake_mask** register to set the valid wake-up sources.

- GPIO, countdown timer, mouse, keyboard, sensor hub, SPI, I2C, PMU (timer, GPIO level, energy4CPU, energy4TX, endoflife, brownout), and wakeup receiver (WURX).

All selections about how to wake need to be programmed before the transition into the low power state is triggered. A transition into hibernation is triggered by setting PSEQ ctrl0.req_hibernate_state and a transition into Retention is triggered by setting ctrl0.req_retain_all_state. Prior to triggering the transition into the low power state, the user application should first disable all interrupt sources except the PSEQ at_apb_pseq's interrupt. Immediately after triggering the transition into the low power state, the user application should place the CPU in wait for interrupt (WFI) mode.

- Wake from timer is done by setting either **hib_wake_mask**.watch_cntdown or **retain_all_wake_mask**.watch_cntdown and programming **init_count_down** register to a non-zero value. This counter operates at 32 kHz and will trigger the appropriate power up sequence when it reaches zero.
- Wake from GPIO is done by setting either **hib_wake_mask**.*watch_gpio* or **retain_all_wake_mask**.*watch_gpio* and programming **hib_wake_mask**, **gpio_wake_type, gpio_wake_pol**, and/or **gpio_wake_both_edges** registers.
- Wake from activity on keyboard and/or activity on mouse is done by setting **hib_wake_mask**.*watch_ksm*, r**etain_all_wake_mask**.*watch_ksm*, **hib_wake_mask**.*watch_qdec*, **retain_all_wake_mask**.*watch_qdec*. The corresponding cores (at_apb_ksm for keyboard and at_apb_qdec for mouse) will have to have been configured for correct operation prior to triggering the transition into the low power state. The activity detection does require some additional top level logic to remain powered. If the application does not use these devices or does not use them to generate a wake event, these cores can be powered down indefinitely using **ksmqdec_control** register.
- Wake from sensor hub is done by setting either **hib_wake_mask**.*watch_shub* or **retain_all_wake_mask**.*watch_shub*. The corresponding core (at_apb_pseq) will have to have been configured for correct operation prior to triggering the transition into the low power state. The sensor hub does require some additional top level logic to remain powered. If the application does not use the sensor hub, at_apb_shub can be powered down indefinitely using **sensor_hub_control** register.
- Wake from activity on SPI slave or I2C slave is done by setting **hib_wake_mask**.*watch_spi*, r**etain_all_wake_mask**.*watch_spi*, or

CONFIDENTIAL

hib_wake_mask.*watch_i2c*, **retain_all_wake_mask**.*watch_i2c*. The corresponding cores (at_apb_spi for spi0 slave and at_apb_i2c for i2c0 slave) will have to have been configured for correct operation prior to triggering the transition into the low power state. The activity detection does require some additional top level logic to remain powered. If the application does not use these devices or does not use them to generate a wake event, these cores can be powered down indefinitely using **ksmqdec_control** and **spii2c_control** register.

- Wake from PMU timer and GPIO level events detection is done by setting **hib_wake_mask**.*watch_pmu_timer*, **retain_all_wake_mask**.*watch_pmu_timer,* **hib_wake_mask**.*watch_pmu_lpcomp*, **retain_all_wake_mask**.*watch_pmu_lpcomp.* Wake from PMU battery/energy detectors is done by setting **hib_wake_mask**.*watch*_energy4CPU, **retain_all_wake_mask**.*watch*_energy4CPU*,* **hib_wake_mask**.*watch*_energy4TX, **retain_all_wake_mask**.*watch*_energy4TX*,* **hib_wake_mask**.*watch*_endoflife, **retain_all_wake_mask**.*watch*_endoflife*,* **hib_wake_mask**.*watch*_brownout, **retain_all_wake_mask**.*watch*_brownout*.*

- Wake from wake-up receiver is done by setting **hib_wake_mask**.*watch_wurx1*, **hib_wake_mask**.*watch_wurx0*, **retain_all_wake_mask**.*watch_wurx1*, or **retain_all_wake_mask**.*watch_wurx0*. Wake from a fine tuned wake-up receiver events filtering is done by setting **hib_wake_mask**.*watch_wurx*, **retain_all_wake_mask**.*watch_wurx.* Filtering operation has to be configured prior to triggering the transition into the low power state using **wurx_config1** and **wurx_config2** registers. The wake-up receiver will have to have been configured for correct operation prior to triggering the transition into the low power state. The wake-up receiver does require some circuitry to remain powered. If the application does not use the wake-up receiver, it can be powered down indefinitely using **wurx_config** register.

- Wake from the detection of a connection over the SWD interface is enabled by default. Refer to ARM Debug Interface Specification v5 for more information on the SWD. If either CSYSPWRUPREQ or CDBGPWRUPREQ is set in the SWD interface, a wake will occur. If this behavior is not desired, CSYSPWRUPREQ and CDBGPWRUPREQ can be cleared or PSEQ **ctrl0**.*block_dbg_wake* bit can be set.

The user application, when returning from a lower power state, can determine why the wake occured by querying status. The register **current_real_time** can be used to track how much real time has elapsed. The real time counter operates from CLK_LPC (32 kHz) and will run continuously and is independent of the power state. The register

CONFIDENTIAL

persistent0 is a general scratch pad register available for software use and may be of use during debug. Its contents persist through all power states and the contents have no impact on the hardware.

This core also generates PMU signals to indicate the amount of energy available from the battery or harvester. pmu_status.brownout signal can be used for brownout support. After the SOC is programmed to enter any low power state, the software first checks the status of brownout. If it is asserted, the SOC will enter its preprogrammed low power state. If it is deasserted, indicating not enough energy from the battery or harvester, the SOC will be programmed to enter SoC Off state, gated by enough energy indication. For SOC with harvesting capabilities, it will be brought out of SoC Off when the harvester is restored to enough energy. For SOC without harvesting capabilities, it will stay in SoC Off.

CONFIDENTIAL

2.10 Peripherals and I/O

The following peripherals are supported by the ATM33/e series SoC.

2.10.1 GPIO

There are up to 31 GPIOs available on ATM33/e, depending on the package. Please see Table 2-18 for a complete list of the available GPIOs. In addition to providing general purpose read and write access, the GPIO can also be used to generate interrupts and to wake the ATM33/e from low power states.

When no power is provided, or during power-down state (PWD held high), all GPIOs are held in a high impedance state. Shortly after PWD is deasserted, most GPIOs are still held in a high impedance state, except for P0/P1 are defaulted to input to support SWD, with P0 additionally enabling an internal pull down, and P1 enabling an internal pull up.

Control registers (e.g. output enable, input enable, interrupt type) are set and cleared by independent registers. The readback on these set/clear registers reflect the state of the control registers. For instance, to change the state of a GPIO's output enable, the user can program the corresponding bit in the **OUTENSET** or **OUTENCLR** registers to 1 at any time and the latest programming of the two registers determines whether that GPIO is an output. A readback of either the **OUTENSET** or **OUTENCLR** register will return the same value. The GPIO can toggle at a frequency of up to 4 MHz (clk_mpc/4) as it takes two cycles to write 1 and two cycles to write 0 on an AHB register. The AHB GPIO provides programmable interrupt generation features. Three registers control this

functionality and each register has separate set and clear addresses. Each bit of the I/O pins can be configured to generate interrupts based on these three registers as shown in Table 2-21.

Interrupt_enable	Interrupt_polarity	Interrupt_type	Interrupt feature
0	N/A	N/A	Disabled
1	0	0	Low-level
1	0	1	Falling edge
1	1	0	High-level
1	1	1	Rising edge

Table 2-21 Interrupt Registers

Each register is controlled by a pair of software-controlled registers. One is used to set; the other one is used to clear. After an interrupt is triggered, the corresponding bit in the GPIO **INTSTATUS_CLEAR** register is set. This also causes the corresponding bit of internal GPIOINT[15:0] signal to be asserted. As a result, the combined interrupt internal signal, COMBINT, is also asserted. The interrupt status can be cleared by using an interrupt handler that writes 1 to the corresponding bit of **INTSTATUS_CLEAR** register.

Controlling input enable and pull up enable

The physical pins have both input enable and pull up enable signals. Disabling the input receivers can save power when the state of the GPIO is unknown.

- These two registers can be used to change the state of the input enable.
  - **INENABLE_SET** : to enable the input driver per pin
  - **INENABLE_CLR** : to disable the input driver per pin (reducing power for GPIO whose input is not used)

- Example: Configure P13 to be GPIO[13] and output only
  - Select GPIO[13] on P13 by programming WRPR **PSEL_D** [15:8] to 0.
  - Program GPIO0 **OUTENSET**[13] to 1 and **DATAOUT**[13] to 0/1 as desired.

- Example: Configure P23 to be GPIO[19] and input only
  - Select GPIO[19] on P23 by programming **PSEL_F** [31:24] to 0.
  - Program GPIO1 **OUTENCLR**[3] to 1 and **INENABLE_SET**[3] to 1. The user samples the input data from DATA[3].

CONFIDENTIAL

2.10.2 Inter-Integrated Circuit (I2C)

I2C interface allows software to preload the transaction and then initiates the hardware controller. Software can either poll for completion or respond to the completion interrupt. Head, tail, acknowledge, data are all software programmable. The hardware will serialize and sample incoming data as required by the protocol. The I2C core is an APB slave. Clock stretching is not supported on the I2C cores.

The SoC has two I2C masters/slaves; selectable with bit **CTRL**.mstr, default is master. I2C master generates signals (SCL, SDA) going to slaves and receives data on SDA. I2C Slave receive control signals (SCL, SDA) from external master, and send data through SDA. The I2C core supports clock frequencies between 3.9 kHz-4 MHz by setting **CLOCK_CONTROL**.clkdiv (frequency = backplane_freq/4*(clkdiv+1)), where backplane_freq is the Backplane Clock frequency in MHz, and the maximum I2C clock frequency is 4 MHz. In idle state, the core drives SCL and SDA high.

To Read/Write slave devices use the following steps:

1) Enable clock, toggle reset to I2C core(s) by writing to **APB0_CTRL** register for i2c0, **APB1_CTRL** register for i2c1 under the WRPR1 section of the register specification.
2) Read I2C0 **CORE_ID** register and it should match 32'h4932_4320 (defined in register spec).
3) Select the master interface by setting **CTRL**.mstr bit to 1, default is master.
4) Setup I2C pins by writing to **PSEL** registers in at_apb_wrpr. If you plan to set up two masters, write to appropriate **PSEL** registers.
5) Write to appropriate I2C **TRANSACTION_SETUP** and **CLOCK_CONTROL** registers (clkdiv, ack, tail, head ptr etc). If writing multiple bytes (burst write), make sure head, tail bytes are set correctly.
6) For Writes, place the data in the **OUTGOING_DATA** register.
7) Set **TRANSACTION_SETUP**.go bit, this initiates I2C FSM and drives transactions.
8) There are two ways to signal transaction end:
    a) Polling - Read status using transaction_status register
    b) Interrupt - I2C core generates interrupt which can be used by the CPU. Interrupt can be masked, set, cleared with appropriate registers
9) For reads, readback byte data from **INCOMING_DATA** register.

CONFIDENTIAL

External master devices Read/Write to/from slave use following steps:

1) Enable clock, toggle reset to I2C core(s) by writing to **APB0_CTRL** register for i2c0, **APB1_CTRL** register for i2c1 under the WRPR1 section of the register specification.

2) Read i2c **CORE_ID** register and it should match 32'h4932_4320 (defined in register spec).

3) Select the slave interface by setting **TRANSACTION_SETUP**.mstr bit to 0.

4) Setup i2c pins by writing to **PSEL** registers in at_apb_wrpr.

5) I2C slave records subsequent transactions once it's received; stores R/W status in **TRANSACTION_STATUS** register, address in **INCOMING_DATA1** register. For writes, write data is sent to the **INCOMING_DATA** register.

6) I2C slave then sends an interrupt (**INTERRUPT_STATUS**.interrupt1) to the CPU core indicating a new transaction is recorded.

7) CPU starts by reading **TRANSACTION_STATUS**.s_rnw bit to determine transaction type; read or write.

8) For Writes, CPU reads **INCOMING_DATA1** for address and **INCOMING_DATA** register for data, and writes data to respective memory address.

9) For Reads, CPU read **INCOMING_DATA1** for address and preload **OUTGOING_DATA1** register with read data from respective address, I2C slave then sends read data out serially on SDA. Note CPU operations to have read data ready, need to be done before the next I2C toggling edge after read address is received. After master send id and write address, then it must issue a restart to switch direction to a read.

10) Address auto increment is supported for back-to-back read/write transactions.

11) For read transactions followed by writes, the master needs to issue a stop condition before switching directions.

12) For write transactions followed by reads, master can either issue a restart or stop-start condition before switching directions. Reads to the same address after writing, the master needs to re-enter the address again.

13) There are two ways to signal transaction recorded:
    a) Polling - Read status using transaction_status register

CONFIDENTIAL

b) Interrupt - Core generates int which is used by the CPU core to finish transactions. Interrupt can be masked, set, cleared with appropriate registers

14) For chip wake-up from LP modes based on I2C slave activity, Master need to send read **CHIP_ID** command and keep polling until slave respond is matching. Applies to i2c0 slave only.

2.10.3 Serial Peripheral Interface (SPI)

SPI interface allows software to preload the transaction, initiate it, and then either poll for completion or respond to the completion interrupt. Opcode, transaction type, data, and number of bytes are all software programmable. The hardware will serialize and sample incoming data as required by the protocol. The SPI core is an APB slave.

The SoC has two SPI masters/slaves; selectable with bit **CTRL**.mstr, default is master. SPI Masters provide control signals (SCLK, CSN, MOSI) to external slaves, and receive data through signal MISO. SPI Slaves receive control signals (SCLK, CSN, MOSI) from external masters, and send data through MISO. SPI masters and slaves support all four SPI modes through CTRL.cpol and CTRL.cpha bits, with default set to Mode-0 (CPOL=0, CPHA=0) and supports SPI master clock frequencies between 7.8kHz-8MHz by setting **TRANSACTION_SETUP**.clkdiv (frequency = backplane_freq/2*(clkdiv+1)), where backplane_freq is the Backplane Clock frequency in MHz, and the maximum SPI master clock frequency is 8 MHz.

To Read/Write slave devices use following steps:

1) Enable clock, toggle reset to SPI core(s) by writing to **APB7_CTRL** register for spi0, **APB13_CTRL** register for spi1 in WRPR0 section of the register specification.

2) Read SPI **CORE_ID** and it should match 32'h5350_4920 as defined in Register Specification.

3) Select the master interface by setting **CTRL**.mstr bit to 1, default is master.

4) Setup SPI pins by writing to **PSEL** registers in at_apb_wrpr. If you plan to set up two slaves, write to appropriate pin selection registers.

5) Write to **TRANSACTION SETUP** register: opcode, clkdiv, num_data_bytes, dummy_cycles.

CONFIDENTIAL

6) Set **TRANSACTION_SETUP**.start bit, this initiates SPI FSM and drives transactions.

7) For Writes, place the data to write in the register: **DATA_BYTES_LOWER, DATA_BYTES_UPPER.**

8) There are two ways to signal transaction end:

    a) Polling - Read status using **TRANSACTION_STATUS** register

    b) Interrupt - Core generates interrupt which can be used by the CPU. Interrupt can be masked, set, cleared with appropriate registers

9) For reads, readback data from **DATA_BYTES_LOWER, DATA_BYTES_UPPER** registers respectively.

10) Burst transactions are done by setting **TRANSACTION SETUP**.csn_stays_low starting at the first read transaction except at the last transaction. For subsequent transactions starting at the second transaction, the lsb byte is stored in **TRANSACTION_STATUS**.opcode_status, subsequent bytes are stored in **DATA_BYTES_LOWER**, **DATA_BYTES_UPPER** respectively.

External master devices Read/Write to/from slave use following steps:

1) Enable clock, toggle reset to SPI core(s) by writing to **APB7_CTRL** register for spi0, **APB13_CTRL** register for spi1 in WRPR0 section of the register specification.

2) Read SPI core_id and it should match 32'h5350_4920 as defined in Register Specification.

3) Select the master interface by setting **CTRL**.mstr bit to 0.

4) Setup SPI pins by writing to **PSEL** registers in at_apb_wrpr.

5) Supported commands opcodes;

    a) 0xE0 -- for slave dummy cycles configuration

    b) 0x60 -- for read from slave

    c) 0x20 -- to write to slave

    Read/Write Opcodes are configurable by setting **CTRL**.s_rd_opcode and **CTRL**.s_wr_opcode respectively.

CONFIDENTIAL

6) For external masters with high spi clock frequencies during read, the master must first configure the slave to insert dummy cycles. External master uses slave config command, to set number of dummy cycles chosen for system and spi clocks frequencies. Config command structure = 1byte opcode (0xE0) + 1byte number of dummy-cycles. For dummy_cycles calculation, use formula ((Fsclk*48cycles)/Fpclk). Ex. for 8 MHz spi clock frequency, dummy_cycles=24.

7) SPI slave records subsequent transactions once it's received; for Read/Write commands, stores the opcode in the **OPCODE_STATUS** register, address in **DATA_BYTES_UPPER** register. For Write, write data is sent to **DATA_BYTES_LOWER** register.

8) SPI slave then sends interrupt (**INTERRUPT_STATUS**.interrupt1) to CPU indicating a new transaction is recorded.

9) CPU starts by reading the **OPCODE_STATUS** register to determine transaction type; read or write.

10) For Writes, the CPU reads **DATA_BYTES_LOWER** for address and DATA_BYTES_UPPER register for data and writes data to respective memory address.

11) For reads, CPU reads **DATA_BYTES_LOWER** for address and preload **DATA_BYTES_UPPER** register with read data from respective address, SPI slave then sends read data out on MISO. Note CPU operations to have read data ready, need to be done before the next SPI toggling edge after read address is received. Otherwise, the external master must insert dummy cycles.

12) Address auto increment is supported for back-to-back read/write transactions.

13) There are two ways to signal transaction recorded:

    a) Polling - Read status using **TRANSACTION_STATUS** register

    b) Interrupt - Core generates int which is used by the CPU core to finish transactions. Interrupt can be masked, set, cleared with appropriate registers

14) For chip wake-up from LP modes based on spi slave activity, Master needs to send a read **CHIP_ID** command and keep polling until slave response is matching. Applies to spi0 slave only.

CONFIDENTIAL

]2.10.4 Quad serial peripheral interface (QSPI)

There is one quad SPI master port and it is intended to be connected to a flash device at up to 48 MHz (when needed). The core is an AHB slave. It generates control/data signals (clk, cs, d[3:0]), which are used to connect to external flash.

This QSPI supports the following features:

1) Programmable commands and modes (Single/Quad data) to support multiple flash vendors.
2) 4/8/16/32/48 MHz speeds supported - Programmed through QSPI **remote_ahb_setup** register.
3) Cacheable via the CPU subsystem's I-cache.
4) Multiple ways to access flash memory - Single/Quad APB, Single/Quad AHB. This helps with Flash bring-up, debug and faster access. To Read/Write flash devices with QSPI, use the following steps: (Also refer to vendor provided datasheet for opcodes, ID values, register fields, definitions etc).

Single SPI APB:

1) Enable clock, toggle reset to SPI2/QSPI core(s) by writing to **APB11_CTRL**(SPI2), **APB15_CTRL**(QSPI) register in WRPR.

2) Enable clock, toggle reset to PSEQ by writing to **APB8_CTRL**(PSEQ). Turn on *clkhpc* bit (32 MHz) by writing to the **XTAL_BITS1** register.

3) Setup QSPI pins by writing to **PIN_SELECTION** registers in at_apb_wrpr.

4) Read spi2/qspi **core_id** and it should match 32'h5350 4920, 32'h5153 5049 respectively as defined in Register Specification.

5) Read flash id (RDID) by setting appropriate **QSPI** registers in SPI2. Make sure it matches vendor provided values. Vendors also provide memory type, density when *num_data_bytes* is set to 3.

6) To write to external flash:
    a) Set Enable (WREN) opcode (refer to flash vendors' documentation) in transaction_setup register in SPI2 (opcode, clkdiv, num_data_bytes, dummy_cycles).
    b) There are 2 ways to signal transaction end
        i) Polling - Issue Read status (RDSR) opcode to read from flash.
        ii) Interrupt - Core generates int which can be used by M33 core. Interrupt can be masked, set, cleared with appropriate registers

CONFIDENTIAL

c) Set Page Program (PP) opcode in transaction_setup register in SPI2 (opcode, clkdiv, num_data_bytes, dummy_cycles). Wait for transaction to complete

7) To read from flash:
   a) Set Enable (WREN) opcode in transaction_setup register in SPI2 (opcode, clkdiv, num_data_bytes, dummy_cycles).
   b) Wait for transaction to end
   c) Set Read (READ) opcode in transaction_setup register in SPI2 (opcode, clkdiv, num_data_bytes, dummy_cycles). Wait for transaction to complete

QSPI Single/Quad AHB:

Using QSPI AHB with Cache, Hyper (32 MHz) and high performance provides the best flash performance.

1) Do steps 1-5 from Single SPI APB.
2) Setup Quad mode in external flash memory:
3) Setup QSPI **remote_ahb setup** registers (1-6), and **transaction_setup** register. Special bits need to be set for following:
   a) Enable Cache bit for faster Flash reads
   b) Enable slip_half_cycle for high performance (for devices that allow the master to send address in Quad mode).
4) Write/Read flash memory.

To bring up flash and get the best performance, follow these steps:

1) Read the flash datasheet and gather opcodes, register definitions, addresses etc.

2) Setup **pin_selection**_x for connection to external flash. Multiple configurations are available based on the package.

3) Setup Single SPI (spi2) access by writing to remote_spi_has_control bit. Read ID, write/read memory. Run at different speeds and make sure the flash is working. This is for initial bring-up.

4) Setup Quad mode by writing QE bit in the status register of the flash. Read ID in Quad mode, write/read memory. This is to ensure Quad SPI works correctly. Try setting Hyper, enable_performance_mode bits and repeat.

5) Setup AHB by writing to *remote_ahb_qspi_has_control* bit:
   a) Read ID, write/read memory.

CONFIDENTIAL

b) Enable Cache and repeat step a. Notice the improved read time after a few flash reads.
c) Set Hyper, *enable_performance_mode* bits and repeat.

2.10.5 Universal Asynchronous Receiver/Transmitter (UART)

There are two UART cores with hardware flow control present on the ATM33/e. UART0 is configured (by Atmosic software) to be used for HCI type applications with flow control and UART1 primarily for debug messaging, therefore without flow control.

The UART is an APB slave. The functionality, except for the Atmosic's modifications, is covered in ARM's Cortex-M System Design Kit Rev r1p0. The UART core supports standard baud rates of 9600, 19200, 38400, 57600, 115200, 460800, 1000000 and 2000000 by setting the **bauddiv** register. It only supports one stop bit and no parity bits. The modifications implement Tx/Rx buffers and the hardware flow feature (nUARTCTS and nUARTRTS). In this implementation, the Rx and Tx are each equipped with a 16-deep FIFO buffer. The Rx FIFO acts as a buffer for the incoming data bytes. The CPU is interrupted when the RX FIFO has waited long enough to have received **rx_lwm** items , starting from the first item received. In addition, when the Rx FIFO is occupied with 13 bytes and beyond, the output nUARTRTS is asserted low to warn the transmitting uart. The Tx FIFO acts as an outgoing buffer so that the CPU can fill up multiple bytes of data at a time and go to attend other tasks while the slow transmission is draining the buffer. The CPU is interrupted when the TX FIFO has been drained to a low water mark set in **tx_lwm**. The constraint on this register is (1<= **tx_lwm** <= 16). An interrupt is issued if the number of available spaces in the FIFO has fallen below the water mark. In addition, the Tx is stopped from sending a new byte when the input signal nUARTCTS is asserted low.

The hardware flow control signals can be overridden with software-controlled registers **hw_flow_ovrd**. Furthermore, for debugging purposes, the user can:

- find the number of free spaces in the RX FIFO and TX FIFO in **rx_fifo_spaces** and **tx_fifo_spaces** respectively.
- see the status of the hardware flow control signals in bit 5 of **state** register for nUARTCTS and bit 4 for nUARTRTS.

2.10.6 Pulse Density Modulation (PDM) Digital Microphone

The PDM-to-PCM block (PDM2PCM) interfaces to a digital pulse-density-modulation (PDM) microphone. The SoC sends the clock to the PDM microphone and the PDM

CONFIDENTIAL

microphone outputs a 1-bit stream to the PDM2PCM block. The PDM2PCM block converts this 1b stream into Pulse-Coded-Modulation (PCM) samples at a given sample rate.

The PDM microphone usually encompasses a transducer (to convert sound waves to electrical), a preamplifier and a sigma-delta ADC which converts this amplified signal to a 1-bit stream. For a stereo microphone, the Left (L) and Right (R) channels are interleaved on the rising and falling edges of the PDM clock.

The specifications for the PDM2PCM block is shown in Table 2-22.

PCM Sample Rates	6, 8, 12, 16, 24, 32, 48, 64 kHz
PCM Bit Width	16 bit, 20 bit, 24 bit
PDM Clock Rates	Programmable between 500 kHz to 3 MHz
Stereo mode support	Yes
High Pass Filter	• Programmable order (1st, 2nd or 3rd) • Programmable BW • Fast initialization to reduce transient
Digital Gain Control	-30 dB to 30 dB in 0.5 dB steps
Stop band attenuation	60 dB

Table 2-22 PDM Block Specifications

CONFIDENTIAL

The SoC has two PDM datapath cores to support Stereo and Mono operational modes. The PDM core is an APB slave.

Each core has a FIFO array for AHB access. AHB FIFO implemented using a ping-pong buffer (64x32). 64 entries are split into 4 arrays -bufferA to bufferD. PDM outputs are written starting at bufferA. Each entry is 32-bit which can be 2- to 16-bit audio samples or single 20/24-bit samples with options for packing and swapping samples. AHB access is faster and data can be DMA'd into SRAM or Flash based on application. Each buffer has an interrupt to indicate that the buffer is full and needs to be emptied, and an error interrupt to indicate that a write occurs before the buffer is being released by software.

To setup PDM core use the following steps:

1) Enable clock, toggle reset to PDM core(s) by writing to **APB0_CTRL** register for spi0, **APB1_CTRL** register for spi1 in WRPR2 section of the register specification.

2) Read PDM **CORE_ID** and it should match 32'h5044_4d20 as defined in Register Specification.

3) Setup PDM (clk, in) pins by writing to **PSEL** registers in at_apb_wrpr.
4) Configure PDM datapath(s) parameters for desired setting in **CONTROL**, **FILTER_CONFIG**, **FILTER_CONFIG1**, **GAIN_CONTROL_CONFIG** registers.
5) Set **CONTROL**.enable_dp bit, this initiates PDM datapath and drives PCM output.

6) FIFO supports 2 interrupts (full, error) and they can be set up individually.

7) Core generates buffers full, buffer write error interrupts which can be used by the CPU. Interrupts can be masked, set, cleared with appropriate registers.

CONFIDENTIAL

2.10.7 Pulse Width Modulation (PWM)

The Pulse Width Modulation (PWM) has eight independent output channels. Each channel has frequency and duty cycle options. All channels can be started at exactly the same time if synchronization is required. The core is an APB slave. Each of the eight channels are identical but have their own configuration registers.

A channel is enabled by setting the **PWM[0-7]_CTRL.**ok_to_run bit. If channels need to be started simultaneously, the **PWM_CTRL**.alt_ok_to_run should instead be used. Each channel can optionally be inverted and has an idle polarity option.

Each channel can operate in one of four modes selectable by setting **PWM[0-7]_CTRL**.mode

1) Continuous mode. Operation in this mode is done by setting register **PWM[0-7]_DUR**.hi_dur[15:0] and **PWM[0-7]_DUR**.lo_dur[15:0] that determine the durations of high and low signal states in clock cycles. Together define the “frame”. In continuous mode the signal frame is endlessly repeated. If registers’ values are changed, they are applied to the next signal frame and not immediately.

2) Counter mode. Operation in this mode is done by setting **PWM[0-7]_DUR,** and **PWM[0-7]_CFG**.frame_count[13:0] register that determines the number of signal frames to define a frame group. The specified number of signal frames are outputted and then the transmission stops.

3) IR mode. Operation in this mode is similar to counter mode except that it repeats. Any register changes are applied during the next frame group.

4) IR FIFO mode. Similar to IR mode. Uses two static registers for two carrier frequencies duty-cycles; **FIFO_CARRIER1_DUR** and **FIFO_CARRIER2_DUR**, such that users can configure two carrier frequencies.The control parameters for each frame group; frame_count, freq_sel, and IR_region_sel are stored in a FIFO. To push a new entry into the fifo, the user writes the above registers to specify (hi_dur[15:0], lo_dur[15:0]) of two carrier frequencies to choose from. Then write to the 16-bit register **FIFO_DATA[15:0]** which encapsulates the dynamic parameters as follow:

    a) fifo_data[15] = freq_sel

    b) fifo_data[14] = IR_reqion_sel

    c) fifo_data[13:0] = frame_count[13:0]

    Software writes to and maintains the FIFO. As long as FIFO is not empty, the hardware keeps pulling frame group parameters. Either the CPU or DMA can be the master supplying entries into the FIFO. An option to stop the channel after the last command in fifo is executed can be used when setting **FIFO_CFG**.ch_stop fifo config bit. There is a set of status bits and interrupt bits associated with this FIFO mode (defined in register spec).

All channels support all modes of operation. One FIFO buffer is time-shared among all 8 channels, and so, IR FIFO mode is only functional in the channel equipped with the FIFO buffer.

To setup PWM channel use the following steps:

1) Enable clock, toggle reset to PWM core(s) by writing to **APB6_CTRL** register in WRPR0 section of the register specification..
2) Read PWM **CORE_ID** and it should match 32'h4014_6FF4 (defined in register specification).
3) Setup PWM pins by writing to **PSEL** registers in at_apb_wrpr.
4) Setup channel mode for selected channel(s). Based on selected mode, setup channel frequency, duty-cycle, number of frames, FIFO parameters.

CONFIDENTIAL

5) Set **PWM[0-7]_CTRL.**ok_to_run bit, this initiates PWM FSM and drives channel output.

With suitable divider values programmed, the core can get to within 5% of the standard remote frequencies (36 kHz, 38 kHz, 40 kHz, 56 kHz).

On the high frequency side it can operate up to 10 MHz.

2.10.8 Quadrature Decoder for Mouse Inputs (QDEC)

The QDEC is a 3-axis (x,y,z) quadrature decoder. Each decoder is independent and requires 2 input pins. QDEC continuously updates the internal integrators and their values can be read by software.

The core is an APB slave and supports QDEC with 3-axis (x,y,z) in multiple configurations. Each axis requires 2-input pins which can be setup using **pin_selection**_x registers in WRPR.

QDEC provides the following registers per axis:

- Setup register (polarity, purge values)
- QDEC core accumulates the individual axis movements and writes into accumulator registers.
- Error values are logged in error count registers
- Accumulator, error purge to clear out accumulator, error values respectively.

To setup QDEC use following steps:

1) Enable clock, toggle reset to QDEC core by writing to apb12_ctrl register in WRPR.
2) Read qdec core_id and it should match 32'h51444543 (defined in register spec).
3) Setup qdec pins by writing to WRPR **pin_selection**_x registers to set up all 3-axis.
4) Set polarity, purge bits in **ctrl0** register.
5) Read accumulator, error register per axis to compute the direction, spin.
6) Clear registers if required.

CONFIDENTIAL

2.10.9 Keyboard Scan Matrix Controller (KSM)

The KSM is a keyboard scanner which supports multiple keyboard sizes up to 18 rows or 18 columns, limited by the total number of available GPIOs in the ATM33/epackage (7x7: 30 combined rows + columns, 5x5. Key pressed and key released events can optionally require that the key be pressed or be released across multiple consecutive scans. Up to 12 simultaneous key events can be tracked.

A portion of the KSM can continue to scan during low power states. The key pressed or key released event can also optionally wake the ATM33/e from a low power state. Key events are not lost when the ATM33/eis in a low power state. Key events are packetized and written into a FIFO via the hardware for later reading by the software.

To setup KSM use the following steps:

1) Enable clock, toggle reset to KSM core(s) by writing to **apb10_ctrl** register in WRPR.
2) Read KSM **core_id** and it should match 32'h57525052 (defined in register specification).
3) Setup KSM pins by writing to **pin_selection** registers in at_apb_wrpr.
4) Setup **matrix_size, time_param0**, and **time_param1** registers.
5) Set go, consecscan bits in ctrl0 register, this initiates KSM scan and starts key capture.
6) There are 2 ways to signal transaction end
    a) Polling - Read status using keyboard_packet register empty bit.
    b) Interrupt - KSM core generates interrupt which can be used by CPU. Interrupt can be masked, set, cleared with appropriate registers
7) FIFO supports 2 interrupts (data, full) and they can be set up individually.
8) Read keyboard packet register which contains pressed/released bit, row/col entry.

2.10.10 Analog Comparator

A 16-level analog comparator provides an ultra-low power approach to sense an analog input signal from sensors.

CONFIDENTIAL

2.11 General-purpose ADC (GADC)

The Application ADC, also known as General-purpose ADC (GADC), enables applications to digitize an external analog signal or certain internal signals (VBAT, VSTORE, AVDD1, temperature). P4, P5, P6, P7 can be configured to be single-ended inputs while P4 and P5 or P6 and P7 pair-wise can be configured to be differential inputs.

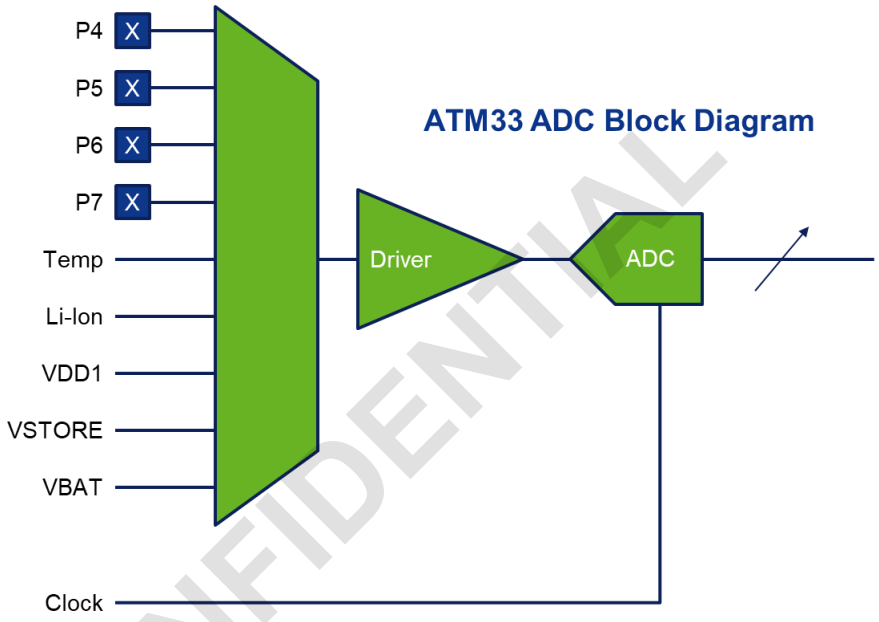


Figure 2-19 Application ADC Block Diagram

CONFIDENTIAL

Pameter	Conditions	Min	Typ	Max	Unit
Resolution			11		Bits
Effective Number of Bits (ENOB)			9		LSBs
Sampling Rate	Single channel, no averaging		2		MS/s
Input Signal Range	VDDIO = 3.3V except li-ion	0		3.2	V
	VDDIO < 3.3V	0		VDDIO	V
SNDR			55		dB
INL				4	LSB
DNL				4	LSB

Pameter	Conditions	Min	Typ	Max	Unit
Input Capacitance			400		fF
Input Capacitance			400		fF
Measurement Accuracy	Temperature		TBD		°C
	Single-ended		10		mV
	Differential-ended		20		mV
	Li-on battery		35		mV

Table 2-23 Application ADC Electrical Parameters

Each channel is one of the possible combinations of actual input mux selection. Each channel has different gains in its signal path which can be selected with the channel-specific gain_sel registers. The gain settings and corresponding input ranges for each channel are summarized in Table 2-24.

watch_channel Set Bit	gain_sel*	Description	Port (7x7 mm Package)	Range (Min, Max)	Resolution (V/LSB)
0	N/A	NOT USED	N/A	N/A	N/A
1	0	VBAT 3.0 V	vin_vbat	0, 3.2	3.91E-03
	1	VBAT 1.5 V	vin_vbat	0.3, 1.8	1.95E-03
2	0	VSTOR 3.0 V	vin_vstor	0, 3.2	3.91E-03
	1	VSTOR 1.5 V	vin_vstor	0.3, 1.8	1.95E-03
3	1	CORE SUPPLY 1.0 V	vin_vdd1	0, 1.5	1.95E-03
4	1	Temperature	iin_temp	-40, 85	0.40 (C/LSB)
5	0	Ext differential mode 3.0 V	P4/P5	-3.0 , 3.0	3.91E-03
	1	Ext differential mode 1.5 V	P4/P5	-1.5, 1.5	1.95E-03
	2	Ext differential mode 0.75 V	P4/P5	-0.75, 0.75	9.77E-04

CONFIDENTIAL

watch_channel Set Bit	gain_sel*	Description	Port (7x7 mm Package)	Range (Min, Max)	Resolution (V/LSB)
	3	Ext differential mode 0.375 V	P4/P5	-0.375, 0.375	4.89E-04
6	0	Ext differential mode 3.0 V	P6/P7	-3.0 , 3.0	3.91E-03
	1	Ext differential mode 1.5 V	P6/P7	-1.5, 1.5	1.95E-03
	2	Ext differential mode 0.75 V	P6/P7	-0.75, 0.75	9.77E-04
	3	Ext differential mode 0.375 V	P6/P7	-0.375, 0.375	4.89E-04
7	0	Ext pos single-ended mode 3.0 V	P4	0 , 3.01	3.91E-03
	1	Ext pos single-ended mode 1.5 V	P4	0, 1.5	1.95E-03
8	0	Ext pos single-ended mode 3.0 V	P5	0 , 3.0	3.91E-03
	1	Ext pos single-ended mode 1.5 V	P5	0, 1.5	1.95E-03
9	0	Ext neg single-ended mode 3.0 V	P6	0 , 3.0	3.91E-03
	1	Ext neg single-ended mode 1.5 V	P6	0, 1.5	1.95E-03
A	0	Ext neg single-ended mode 3.0 V	P7	0 , 3.0	3.91E-03
	1	Ext neg single-ended mode 1.5 V	P7	0, 1.5	1.95E-03
B	2	Li-ion battery		0, 4.2	5.86E-03
C	N/A	INPUT SHORT	N/A	N/A	N/A

Table 2-24 Application ADC Channels Summary

*Each channel has an individual gain_sel register (Eg. for channel 1: ch1_gain_sel)

From the SoC point of view, the GADC core is an APB slave. This core generates the GADC control signals; applies offset, gain, and averaging functions; and contains a FIFO for the CPU to get access to the post processed data.

For the digitizing flow to work, the GADC needs to first be powered up. This is done via the PSEQ **config** register. The steps are:

CONFIDENTIAL

1) Short the supply head switch via **gadc_config**.*gadc_cutvdd_b*
2) Then release reset and set the enable via **gadc_config**.*gadc_rstb* and **gadc_config**.*gadc_en*.

The GADC channels to be monitored are selected via **ctrl**.*watch_channels* with programmable gain setting of **gain_config**.*{}_gain_sel*. The brackets indicate the channel the gain setting should be applied to. If multiple input channels are selected, the input mux will be set in a round-robin schedule. The description of each (**ctrl**.*watch_channels*) are in each row of watch_channels and gain_sel in Table 2-24. If the li-ion battery channel is enabled, **gadc_ctrl**.*li_en* should be set to 1 before operation. When the li-ion voltage is not being measured, this register should be set to 0. The core contains a single output interrupt with different sources. Each source is maskable, clearable, and settable independently. The interrupt sources are:

- Hardware written to FIFO, may be of use in cases where **ctrl**.*wait_amount* is long enough for the CPU to pull from the FIFO before GADC writes to FIFO.
- FIFO overrun.

The samples in the FIFO contain both the channel ID and the GADC sample. The FIFO sample is 21-bit wide with the upper nibble being the channel ID and the next 12 bits being the averaged GADC sample and the last 5-bit as an exponent. The GADC sample in FIFO is signed.

To convert raw GADC samples to real values (voltage or temperature), averages a fixed number of samples, and applies chip-specific calibration data that is stored in OTP. The raw sampling rate of GADC is 2MSPS, but the data throughput to the user depends on the number of channels on measurement, the speed of FIFO reading in software, etc. For the case of one channel measurement with no loading on the cpu, the complete procedure of making a measurement in software can take up to approximately 200 µs. There is a tradeoff between the measurement time and accuracy when using hardware averaging, and by increasing the averaging amount, the measurement time will be longer. As an example, with averaging amount of 3, the measurement will take up to 300 µs.

2.12 Register Specification Document

The various registers of ATM33/e series are documented in the Atmosic ATM33/e Register Specification available on the Atmosic Support website.

CONFIDENTIAL

Reference Documents

Title	Document Number
ARM Cortex-M33F Technical Reference Manual available from https://www.arm.com	NA
Atmosic PIN MUX Software Tool (see README file in the latest SDK v5.2.0 or later)	NA
ATM33 Series Datasheet	ATM33-DS
ATM33e Series Datasheet	ATM33e-DS
ATM33e Series Energy Harvesting Application Notes	AT33e-ANEVH
ATM33e Series Energy Harvesting EVK Quick Start Guide	ATM33e-QSGEHV
ATM33/e Series Evaluation Kit User Guide	ATM33_e-UGEVK
ATM33/e Series EVK Power Consumption Evaluation User Guide	ATM33_e-UGPCE
ATM33/e Series Hardware Design Guide	ATM33_e-DGHW
ATM33/e Series Pulse Width Modulation Application Note	ATM33_e-ANPWM
ATM33e Series PV/RF Harvesting with Li-ION Charging Application Note	ATM33e-ANPVRFHV
ATM33/e Series Reference Manual	ATM33_e-RM
ATM33/e Series Register Specification	ATM3_e-RSB0
ATM33/e Series Release Notes	ATM33_e-RNHWB0
Understanding Low Power Modes for ATM33/e Series White Paper	ATM33_e-WPLP

CONFIDENTIAL

Revision History

Date	Version	Description
March 10, 2023	0.57	Updated Figure 2-18 Wakeup Receiver Block Diagram.
March 6, 2023	0.56	Updated MCU and Memory, Power Modes, Table 2-13 PMU External Pins, Table 2-18 Total Number of I/Os, Figure 2-6 Non-rechargeable or Rechargeable Battery on VBAT, Internal I/O Supply, No Harvesting Capability through Figure 2-14 Li-Ion Battery or Storage Device on VBATLI, Internal I/O Supply, Non-RF Harvesting Source, Wakeup Receiver, Serial Peripheral Interface (SPI), Table 2-23 Application ADC Electrical Parameters. Added OTP Memory section, Figure 2-17 Wakeup Receiver Application Block Diagram, Figure 2-18 Wakeup Receiver Block Diagram.
January 13, 2023	0.55	Updated Supported Battery Types, VSTORE maximum threshold (ATM3330e only), Table 2-15 PMU Supported Configuration, Table 2-17 All Supported PMU Configurations, Serial Peripheral Interface (SPI) sections.
December 19, 2023	0.54	Updated Figure 2-3 through Figure 2-13 VBATLI to VBAT connection, see also Table 2-13 PMU External Pins, Secure Counters, Universal Asynchronous Receiver/Transmitter (UART), Table 2-23 Application ADC Electrical Parameters. Changed VDD1A to AVDD1 throughout the document. Updated Supported, Battery Types, Battery Level, Internal or External I/O Supply, Inter-Integrated Circuit (I2C), Serial Peripheral Interface (SPI) sections.
September 8, 2022	0.53	Updated MCU and Memory, Table 2-13 PMU External Pins, and last paragraph of General-purpose ADC (GADC) section.
August 31, 2022	0.52	Added Table 2-2 List of APB Slave Cores. Updated General-purpose ADC (GADC) section.
August 17, 2022	0.51	Updated Clocks last bullet, Power Modes bullet 5, Debug Access, Interrupts, Security, PMU configurations, Multiplexed Pin Table PSEQ, GPIO sections. Added Trace Port Interface Unit,
May 9, 2022	0.50	Initial version created

CONFIDENTIAL

ATMOSIC TECHNOLOGIES – DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2022-2023 by Atmosic Technologies. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc. All other trademarks are the properties of their respective holders.

CONFIDENTIAL

Atmosic Technologies | 2105 S. Bascom Ave. | Campbell CA, 95008
www.atmosic.com